

7

Assignment 2 系统篇:Systems and Parallelism

7.1. Assignment 2 Overview

7.1 Assignment 2 Overview 85

实验目标: 学习如何从**系统底层**优化提升**单 GPU 的训练速度**以及如何将训练扩展到**多 GPU**。

每一部分的实验任务

1. 基准测试与性能分析框架
2. Flash Attention 2 Triton 内核编写
3. 分布式数据并行训练
4. 优化器状态分片

文档作者: 喂喂微

文档作者：喂喂薇

在我们进行优化之前,我们首先就得清楚程序或模型哪个地方还不够好,哪个地方还有可提升的空间。一般来讲可优化的点就是时间和空间(对应我们老生常谈的时间复杂度和空间复杂度)。那么我们该如何观察到某一个部分的时间和空间的消耗程度呢?

讲义提供了三种性能评估方式:

1. 使用 Python 标准库进行简单的**端到端基准测试**,以测量前向和后向传递的时间;
2. 使用**NVIDIA Nsight Systems**工具进行计算分析,以了解这些时间在 CPU 和 GPU 操作上的分布;
3. 分析**显存使用情况**。

8.1. 端到端简单性能评估测试

参考资料 8.1

- https://blog.csdn.net/gitblog_00002/article/details/148993434
- https://blog.csdn.net/weixin_40198079/article/details/129726302

为了找到程序的瓶颈点,实现快速迭代优化。我们最好以**脚本化**的方式进行性能评估测试,例如我们可以把需要调节的超参数设置成命令行参数,然后通过脚本自动化地进行性能评估测试。讲义强烈推荐使用 Sbatch 或 Slurm 平台上的**submitit**工具来实现批量扫描。

8.1.1. Slurm 平台上的 submitit 工具简要介绍

submitit 工具也是一种批量调度脚本化的工具,和使用 shell 脚本不同,submitit 和 python 绑定程度更高,它甚至是直接无缝衔接在 python 代码中的。

举个例子,比方说我们要实现一个简单的加法函数。

```
import sys
import time
def add(a, b):
    time.sleep(10) # 模拟耗时
```

8.1 端到端简单性能评估测试	87
8.1.1 Slurm 平台上的 submitit 工具简要介绍	87
8.1.2 前向和反向传播性能评估	89
8.2 Nsight Systems Profiler	90
8.2.1 nsys 简介	90
8.2.2 nvtx	90
8.3 混合精度训练	93
8.3.1 浮点数知识回顾	94
8.3.2 混合精度训练的实现	96
8.4 显存分析	97

```

    return a + b
if __name__ == '__main__':
    # 从命令行读取参数
    a = int(sys.argv[1])
    b = int(sys.argv[2])

    result = add(a, b)

    # 打印结果，Slurm 会把这个输出保存到文件中
    print(f"计算结果是: {result}")

```

之后我们配合 shell 脚本实现。但如果我们使用了 submitit 工具，我们就可以直接在 python 代码中实现。

```

import submitit
import time
# 我们的“菜谱”函数，和之前完全一样，甚至更纯粹
def add(a, b):
    time.sleep(10) # 模拟耗时
    return a + b
# 1. 实例化一个“助理” (Executor)
# AutoExecutor 会自动检测到你正在 Slurm 环境下，并进行配置
executor = submitit.AutoExecutor(folder="slurm_logs")
# 2. 设置“厨房”要求（等同于 #SBATCH 参数）
executor.update_parameters(
    timeout_min=5, # 任务最长运行 5 分钟
    cpus_per_task=1,
    mem_gb=1
)
print("准备向集群提交任务...")
# 3. 直接让“助理”把你的 Python 函数和参数送去做
job = executor.submit(add, 5, 7)
print(f"任务已提交，任务 ID 是: {job.job_id}")
# 4. 直接在 Python 里等待并获取结果
result = job.result() # 这行代码会等待任务完成，然后把返回值给你
print(f"从集群拿回了结果: {result}")

```

在需要做**大量重复实验**时（比如机器学习调参）。

假设我们要计算三组不同的加法：(5,7), (10,20), (100,200)。

```

params = [(5, 7), (10, 20), (100, 200)] # 准备多组参数
# 一句话提交一个任务数组！
jobs = executor.map_array(add, *zip(*params)) # zip(*params) 会把 [(5,7),...] 变成
↔ ([5,10,100], [7,20,200])
print(f"一次性提交了 {len(jobs)} 个任务！")
# 等待所有任务完成，并一次性取回所有结果
results = [job.result() for job in jobs]
print(f"所有任务都完成了，结果是: {results}")
# 输出将会是：所有任务都完成了，结果是: [12, 30, 300]

```

submitit 的一大核心优势是**控制器与工作任务的解耦**，当执行 `job = executor.submit(...)` 时，发生了两件独立的事情：

- **控制器（你的本地脚本）**：它的任务是打包你的函数和参数，生成一个 Slurm 脚本，通过 `sbatch` 命令把它发送给 Slurm 调度器。一旦 Slurm 确认收到（返回一个任务 ID），控制器的主要任务就完成了。之后它做的 `job.result()` 只是一个可选的“等待和接收”动作。
- **工作任务（集群上的脚本）**：Slurm 收到请求后，会在某个计算节点上启动一个全新的、独立的进程。这个进程负责执行你的深度学习函数。它的生死只由 Slurm 和它自己决定，和你的“控制器”脚本没有任何关系。

8.1.2. 前向和反向传播性能评估

首先，通过计时前向和反向传递来对模型进行最基础的性能分析。由于仅需测量时间和内存消耗，因此将采用随机权重和数据。

Keynote 8.1

CUDA 异步调用

当 GPU 去进行矩阵计算的时候，由于 CUDA 默认是**异步执行**，CPU 也会继续执行下面的任务，不会等待 GPU 执行完毕之后再去。这样做充分利用了计算资源，但在测量的时候由于是异步，所以不能够通过 `time.time()` 这种方式直接测量 GPU 计算时间（因为 CPU 会直接跳过 GPU 运算的步骤），要测量需要 `torch.cuda.synchronize()` 强制同步。

评估流程：

- 给定**超参数**（例如层数），初始化模型。
- 生成随机数据批次。
- 运行 w 个**预热步骤**（在开始测量时间之前），然后记录 n 个步骤的执行时间（根据参数选择仅前向或前向和反向传递）。

Keynote 8.2

端到端基准测试里面的预热并非学习率预热，而是希望机器在训练了几个步骤达到稳定之后再去测试时间，否则由于刚开始的随机权重和数据，导致损失值非常大，梯度也非常大，导致训练不稳定，时间测量不准确。

- 对于计时，可以使用 Python 的 `timeit` 模块（例如使用 `timeit` 函数，或者使用 `timeit.default_timer()`，它提供系统最高分辨率的时钟，因此比 `time()` 更适合作为基准测试的默认选项）。
- 每个步骤后调用 `torch.cuda.synchronize()`。

8.2. Nsight Systems Profiler

参考资料 8.2

```
https://github.com/NVIDIA/NVTX  
https://nvidia.github.io/NVTX/python/reference.html#nvtx.start\_range
```

端到端基准测试无法揭示模型在前向传播和反向传播过程中具体消耗时间与内存的环节，因而难以发现特定组件的优化空间。为准确掌握程序**各组件（如函数）**的运行耗时，可采用性能分析工具。执行分析器通过在函数开始和结束时插入监控点来检测代码，从而提供函数级别的详细执行统计（包括调用次数、该函数累计耗时等指标）。

像 CProfile 这样的 Python 性能分析器无法对 CUDA 内核进行性能分析，因为 CUDA 内核在 GPU 上是异步执行的。因此我们将利用 NVIDIA 提供的一个可以通过命令行工具 nsys 使用的性能分析器。

8.2.1. nsys 简介

nsys 全称 **NVIDIA Nsight Systems**，是一个专门记录 CPU、GPU 具体工作的工具，输出的结果类似甘特图、时序图这种，会配合 nvtx 使用清楚标注每一个事件的起始时间。

nsys 的工作流程 (The Workflow)

1. Step 1: 命令行分析 (Profile on the Command Line)

讲义中给出的命令是：

```
uv run nsys profile -o result.nsys.rep python benchmark.py
```

- nsys profile: 这是核心命令，告诉 Nsight Systems “开始录制！”。
- -o result.nsys.rep: 这个参数非常重要。-o 代表 output。它告诉 nsys 将所有录制下来的性能数据保存到一个名为 result.nsys.rep 的文件中。.rep 是报告（report）文件的后缀。
- python benchmark.py: 这是你想要分析的目标程序。

2. Step 2: 查看报告文件 (The .nsys-rep File)

3. Step 3: 图形界面分析 (Analyze in the GUI)

8.2.2. nvtx

NVTX API 本身是一套“空接口”或“占位符”。默认情况下，在你的代码中调用 NVTX 函数**不会产生任何实际操作，也不会带来性能开销**。

它的作用只在当你的程序被一个专门的“**开发者工具（如性能分析器）**”启动时才会被激活。这时，这些 NVTX 调用就会被该工具拦截，并转交（重定向）给工具内部的相应功能来处理。

比如一个运动员在跑道上跑步，而教练在赛道上某个位置（比如 100 米、400 米）打个标记方便后续分析，但这些标记不会对运动员跑步产生直接影响。

由于 NVTX 只定义了调用的规范（比如“标记一个时间范围的开始”），而没有规定具体如何响应，因此不同的开发者工具可以根据自己的需要，自由决定如何利用这些调用信息。例如，有的工具可能用它来计时，有的则用它来触发日志记录。

一些常见工具配合 NVTX 使用的例子

- **打印一条消息到控制台**
- **工具精确记录下每一次 NVTX 调用发生的具体时间点。**当程序运行结束后，工具会将所有这些带有时间戳的事件收集起来，并以时间轴的形式图形化地展示出来。这样可以直观地看到代码中不同标记阶段的执行顺序、持续时间以及它们之间是否存在重叠。
- **统计 NVTX 记录下的时间**
- **作为开关，在 NVTX 调用限定的范围内启用/禁用工具功能**
- **作为一个中转站，将数据转发到其他工具出处理**

NVTX 提供的注解类型

● Markers

基础注解类型，主要传递消息，提供了一些选择**颜色、类别**的参数(在 Ranges 里也有这些可选参数)，配合 Nsight System 这种可视化的性能分析工具使用。

```
nvtx.**mark**(  
    message: str | None = None,  
    color: str | int | None = 'blue',  
    domain: str | None = None,  
    category: str | int | None = None,  
    payload: int | float | None = None,)
```

● Ranges

Ranges 用于标记程序在**一段时间内**的活动，就像一对相关联的标记点（一个开始，一个结束）。它主要用来衡量一个函数或一段代码块的执行耗时。

这也是 cs336 希望我们使用的。

NVTX 提供了两种不同机制的范围，以适应不同的应用场景：

1. 推入/弹出式范围 (Push/Pop Ranges):

- **工作机制：** 这种范围像一个**栈 (Stack)** 一样工作，遵循“后进先出” (Last-In, First-Out) 的原则。当你 ‘Push’ 一个新范围时，它就被推入栈顶；当你 ‘Pop’ 时，最顶端的范围被弹出。
- **核心特点：** 它们必须是**严格嵌套**的，不能交叉重叠。‘Pop’ 操作总是自动与同一线程上最近一次的 ‘Push’ 操作配对。
- **适用场景：** 非常适合标记结构清晰、层层调用的函数或代码块。例如，‘main’ 函数调用 ‘function_A’，‘function_A’ 又调用 ‘function_B’。

```

nvtx.push_range("数据处理总流程")
// 步骤 1: 加载数据
nvtx.push_range("加载数据")
...加载数据的代码...
nvtx.pop_range() // "加载数据"范围结束

// 步骤 2: 处理数据
nvtx.push_range("处理数据")
...处理数据的代码...
nvtx.pop_range() // "处理数据"范围结束

// 步骤 3: 保存结果
nvtx.push_range("保存结果")
...保存结果的代码...
nvtx.pop_range() // "保存结果"范围结束
nvtx.pop_range() // "数据处理总流程"范围结束
主程序结束

```

在 Nsight system 上的结果就是:

- 一个最长的、名为“数据处理总流程”的范围。- 在它**内部**，依次排列着三个互不重叠的短范围：“加载数据”、“处理数据”和“保存结果”。- 你永远不会看到“加载数据”还没结束，“处理数据”就开始的情况。这就是严格嵌套。

2. **开始/结束式范围 (Start/End Ranges):** - **工作机制:** ‘Start’ 操作会返回一个唯一的**句柄 (Handle)**，这个句柄就像一个凭证，唯一对应。你必须在未来的某个时刻调用 ‘End’，并将这个凭证传递给它，才能正确地关闭对应的范围。- **核心特点:** 它们可以**任意重叠**，并且一个范围的开始和结束可以发生在**不同的线程**上。- **适用场景:** 用于标记复杂的、异步的或并行的操作，这些操作的生命周期不是简单的嵌套关系。

主程序线程:

```

...
// 步骤 1: 加载数据
// 步骤 2: 并行处理数据
// 为工作线程 1 的任务创建一个范围，并拿到凭证 handle1
handle1 = nvtx.start_range("并行任务 1")
// 启动工作线程 1，把 handle1 交给它

// 为工作线程 2 的任务创建一个范围，并拿到凭证 handle2
handle2 = nvtx.start_range("并行任务 2")
// 启动工作线程 2，把 handle2 交给它

// 主线程可以继续做其他事，或者等待线程结束...

- ----- 时间流逝 -----

工作线程 1 (在另一个 CPU 核心上):
...执行任务 1 的代码...
// 任务完成，用凭证 handle1 关闭对应的范围
nvtx.end_range(handle1)

```

```
工作线程 2（在另一个 CPU 核心上）：  
...执行任务 2 的代码...  
// 任务完成，用凭证 handle2 关闭对应的范围  
nvtx.end_range(handle2)
```

在性能分析工具的时间轴上，你会看到：

- “并行任务 1”和“并行任务 2”这两个范围是在主线程上**几乎同时开始**的。
- 它们的执行过程在时间上是**重叠**的。
- 它们的结束点发生在各自的工作线程上，并且结束时间**可能不同**。

3. Resources

资源命名：

对于新的线程，在 Nsight 里面分析的结果默认显示 python (TID: 54321) 这种线程名，资源命名就是给这个线程起一个名字方便观测处理。

资源追踪：

是命名的扩展，对于一些标准的比如加锁、解锁的过程，NVTX 只需命名，其他工具可以自动识别并给予、释放资源；

对于一些不标准的，比如自己写的自旋锁，NVTX 除了命名之外，还需要明确指出哪里加锁了，哪里解锁了。

我的结果示例

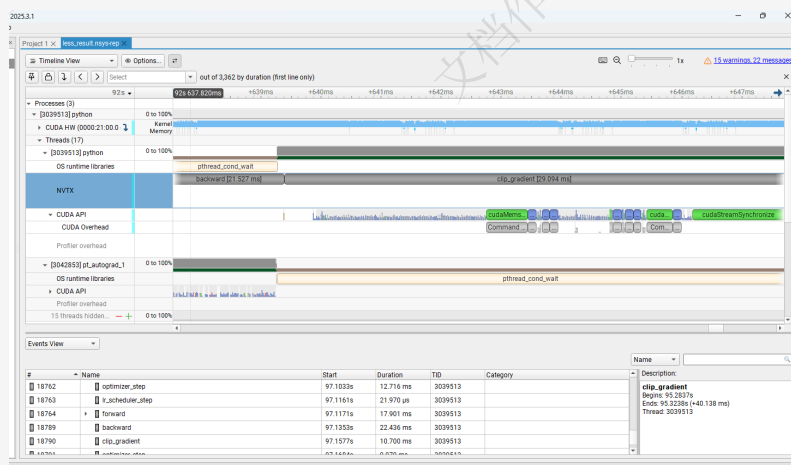


图 8.1: nvtx 结果示例

从结果上来看，没想到裁剪梯度这一步居然这么耗时。。。

8.3. 混合精度训练

到目前为止，在这个任务中，我们一直在使用 FP32 精度——所有模型参数和激活值都具有 torch.float32 数据类型。然而，现代 NVIDIA GPU 包含专门的 GPU 核心（张量核心），用于以较低精度加速矩阵乘法。例如，NVIDIA A100 规格表显示，其在 FP32 下的最大吞吐量为 19.5 TFLOP /秒，而使用 FP16（半精度浮点）或

BF16（脑浮点）时，最大吞吐量显著提高至 312 TFLOP/秒。因此，使用较低精度的数据类型有助于加快训练和推理速度。

然而，若简单地将我们的模型转换为低精度格式，可能会导致模型准确度降低。在此情况下，混合精度训练（Mixed Precision Training）提供了一种解决方案。它允许我们使用较低精度（如 FP16 或 BF16）来计算模型参数和激活值，同时保持 FP32 精度来计算梯度。这不仅提高了训练速度，还避免了精度丢失问题。

总结混合精度训练的特点：

朴素的训练默认使用 32 位浮点数（FP32），而混合精度训练引入了 16 位浮点数（FP16 或 BF16），主要带来两大好处：

1. **减少显存占用**：模型参数、梯度和优化器状态占用的显存几乎减半。这意味着可以用同样的显存训练更大的模型，或者使用更大的批量（batch size）。
2. **加快训练速度**：现代 NVIDIA GPU 内置了 Tensor Cores，专门用于加速 FP16/BF16 的矩阵运算，其理论吞吐量远高于 FP32。使用低精度计算能充分利用这部分硬件性能，带来显著的训练加速（通常能提升 1.5 倍至 3 倍）。

但是，混合精度训练也存在两大挑战：

1. **数据溢出**：直接把所有东西都变成 FP16 是行不通的，因为它的数值范围（约 $6e-5$ 到 65504）太小，容易出现**上溢（Overflow）**变成‘inf’和**下溢（Underflow）**变成‘0’，导致训练崩溃。
2. **下溢（Underflow）**：指一个数因为**太小**（过于接近零），超出了当前数据类型能表示的最小精度范围，结果被强制舍入为**零**。这就像用一把最小刻度是“毫米”的尺子去测量一粒灰尘，由于灰尘太小，你只能记录下它的长度是“0 毫米”。
3. **上溢（Overflow）**：指一个数因为**太大**，超出了当前数据类型能表示的最大范围，结果变成了一个特殊值——**无穷大（‘inf’）**。这就像用一把 30 厘米的尺子去测量一张 1 米长的桌子，你只能说它的长度“超出了尺子的范围”。
4. **舍入误差**：当网络模型的反向梯度很小时，一些 FP32 能够表示的数值可能不能满足 FP16 精度下的表示范围，导致被强行舍入，带来误差。比如 $0.66\dots6$ （32 位）被舍入成 $0.66\dots7$ （16 位）。

Keynote 8.3

溢出是范围问题，它是指数值超出了 FP16 表示的范围；而舍入误差是精度问题，它发生在数值可以被 FP16 表示范围呢，但精度不能够被 FP16 表示。

8.3.1. 浮点数知识回顾

定点数表示小数点位置是固定的，而浮点数则是用科学计数法来表示数值。

IEEE 浮点表示

$$V = (-1)^s \times M \times 2^E$$

- 符号 s : $s=1$ 代表负数, $s=0$ 代表正数
- 尾数 M : 尾数是浮点数的有效精度部分;
e.g. $6.75 = 1.6875 \times 2^2$, 1.6875 就是尾数 M 。尾数隐含以 1 开头, $M=1+f$, 实际存的只有小数点后面的数
- 阶码 E : 决定数值的范围;
值得注意的是在规格化下: 指数位会存在一个偏移, 偏移的好处是为了方便数值大小的比较和避免 0 的不唯一表示。偏移量 $bias = 2^{k-1} - 1$; 比如 8 位指数的偏移量 $bias = 127$ 。
- 最后存储的指数 $E = e() + bias$

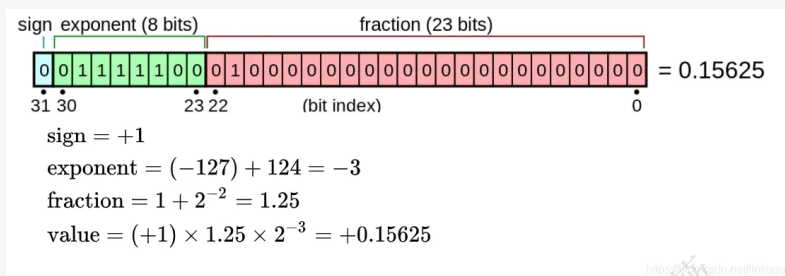


图 8.2: IEEE 浮点表示

半精度浮点数、单精度浮点数和双精度浮点数

- 半精度浮点数 (FP16) 包括 1 位符号位, 5 位指数, 10 位尾数。
- 单精度浮点数 (FP32) 包括 1 位符号位, 8 位指数, 23 位尾数。
- 双精度浮点数 (FP64) 包括 1 位符号位, 11 位指数, 52 位尾数。

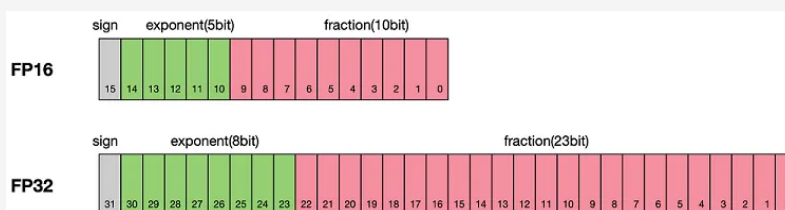


图 8.3: 半精度浮点数、单精度浮点数和双精度浮点数

规格化、非规格化和特殊值

规格化: 指数 E 既不是全 0, 也不是全 1; $E = e - bias$, 对于单精度全 0 $\rightarrow -126$, 全 1 $\rightarrow 127$. $bias = 2^{k-1} - 1$; 尾数部分隐含 1 开头的情况; 大部分都是这种情况。

非规格化: 指数 E 全 0; 阶码 $E = 1 - bias$; 尾数不全 0, 没有隐含的 1, $0.xxxxx \times 2^{-126}$ 为范围 (单精度)。

特殊值: 尾数和指数全为 0 时, 代表 ± 0

指数全为 1, 尾数全为 0 时, 为 $\pm\infty$

当指数全为 1, 尾数非全 0, 为 NaN

8.3.2. 混合精度训练的实现

梯度缩放 (Loss Scaling): 解决数据溢出问题

举个例子，在模型前向传播、计算损失使用 FP16 精度计算完了之后，某个权重计算得到的真实梯度是 ‘1e-6’，如果不使用缩放还使用 FP16 去存储的话，那么因为 ‘1e-6’ 已经不能被 FP16 半精度所表示，它就自动被视为 0，这样就成了梯度消失，学习无效。

而缩放就是在计算得到梯度之后给它乘一个缩放因子，比如 65536，乘了之后的结果就是 FP16 可以表示的了。

Keynote 8.4

在深度学习训练中，尤其是在模型后期趋于收敛时，计算出的梯度值经常会变得非常非常小，比如 ‘1e-6’，‘1e-7’ 等，所以一般缩放因子都比较大，只针对下溢而非上溢出。

权重备份 (Weight Backup): 解决舍入误差

假设你有一个 FP16 格式的权重，值是 ‘0.9824’。现在计算出一个梯度更新量，在 FP32 下是 ‘0.0010001’。但在 FP16 下，这个更新量可能被舍入成 ‘0.001000’。所以 ‘0.9824 + 0.001000’ 在 FP16 下可能是 ‘0.9834’。而最后一位这个小更新就永久丢失了。一次两次没问题，成千上万次迭代后，大量的小更新都丢失了，模型就学偏了或不收敛了。

权重备份的思路就是把权重备份一个 FP32 精度的，每次计算梯度更新权重的时候就用 FP32 的来计算，得到新结果再转换成 FP16，方便下一次计算梯度、激活等操作。

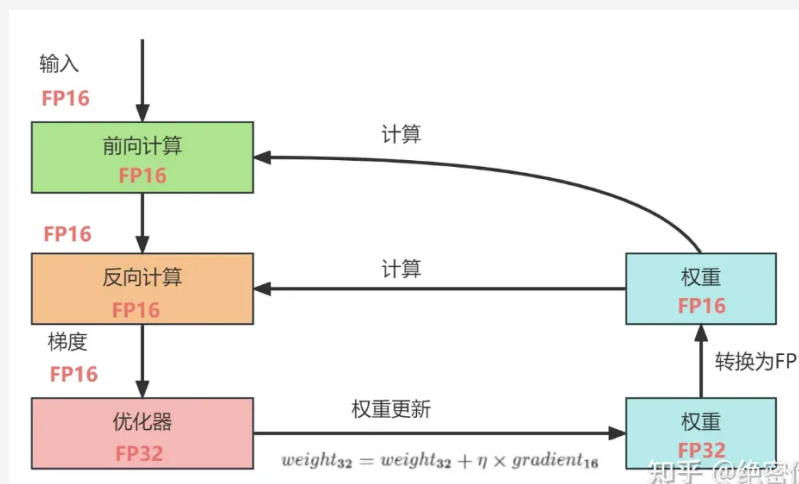


图 8.4: 权重备份

精度累加 (precision accumulated): 解决舍入误差

与权重备份类似，精度累加也是利用了 FP32 精度作为中介来解决舍入误差的问题，区别在于精度累加的过程是在**计算矩阵乘法运算**的时候，计算的结果用 FP32 保存减少舍入误差，然后再转成 FP16 格式进行下一步计算。



图 8.5: 精度累加

Keynote 8.5

事实上，在解决舍入误差的问题上，更新后的 FP32 主权重转换回 FP16 用于下一次计算时，会引入一次新的舍入误差。但这个误差是一次性的、非累积的。真正关键在于，所有微小的梯度更新信息已经被精确地、永久地累积到了 FP32 的主权重上。我们避免了最致命的累积误差问题。

8.4. 显存分析

参考资料 8.3

https://hugging-face.cn/blog/train_memory

此前我们主要探讨了计算的时间性能，接下来将聚焦于**显存**这一语言模型训练与推理中的关键资源。PyTorch 自带强大的内存分析工具 Memory Profiler，可实时追踪内存分配动态。

对于训练和推理大语言模型 (LLM) 来说，显存 (GPU Memory) 和计算的时间性能一样，都是极其宝贵的资源。显存瓶颈常常会导致 “Out of Memory” (OOM) 错误，限制了我们能使用的模型大小和批量大小 (batch size)。因此，精确地分析显存都用在了哪里，是性能优化的关键一步。

Memory Profiler 的核心流程是：

1. 通过代码启动内存历史记录，运行需要分析的目标代码，

```
torch.cuda.memory._record_memory_history(max_entries=1000000)
```

2. 然后将记录到的内存分配快照 (snapshot) 保存成一个 ‘pickle’ 文件。

```
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")
```

3. 停止记录。

```
torch.cuda.memory._record_memory_history(enabled=None)
```

4. 可视化工具分析。

- 使用官方工具：访问图片中提到的在线工具 ‘https://pytorch.org/memory_viz’。
- 加载快照文件：将你本地生成的 ‘memory_snapshot.pickle’ 文件拖拽到这个网页上。
- 解读分析结果：
 - 内存时间线 (Timeline)：你会看到一个图表，显示了总的预留显存 (Reserved Memory) 和已使用显存 (Allocated Memory) 随时间的变化。这可以帮助你快速定位显存峰值出现在哪个阶段。
 - 内存分配详情 (Allocations)：工具会列出每一次具体的内存分配事件。最关键的是，它会提供每一次分配的**大小 (Size)** 和**代码堆栈追踪 (Stack Trace)**。通过堆栈追踪，你可以精确地知道是你的代码中的哪一行（例如，‘model.forward()’ 里的某个特定操作）导致了这次内存分配，从而实现精确优化。

示例代码：

```
import torch
import torch.nn as nn

# 确认你的环境支持 CUDA

if not torch.cuda.is_available():
    print("CUDA is not available. This script requires a GPU.")
else:
    device = torch.device("cuda")

# 1. 定义一个简单的模型
class SimpleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer1 = nn.Linear(in_features=1024, out_features=2048)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(in_features=2048, out_features=512)

    def forward(self, x):
        return self.layer2(self.relu(self.layer1(x)))

# 2. 实例化模型和输入数据，并移动到 GPU
model = SimpleModel().to(device)
# 创建一个需要梯度的输入张量
input_tensor = torch.randn(256, 1024, device=device, requires_grad=True)

# 3. 开始记录内存历史
print("===== 开始内存性能分析 =====")
```

```
torch.cuda.memory._record_memory_history(max_entries=1000000)

# --- 我们想要分析的核心代码段 ---
# 执行一次前向传播
output = model(input_tensor)
# 计算一个标量损失
loss = output.sum()
# 执行一次反向传播
loss.backward()
# -----

# 4. 保存内存快照到文件
snapshot_filename = "simple_model_snapshot.pickle"
torch.cuda.memory._dump_snapshot(snapshot_filename)

# 5. 停止记录
torch.cuda.memory._record_memory_history(enabled=None)

print(f"===== 性能分析结束，快照已保存至 '{snapshot_filename}' =====")
print("请访问 <https://pytorch.org/memory\_viz> 并上传该文件进行分析。")
```

微言大义 8.1

注意这里分析的是 GPU 的显存而非主机的内存。
分析计算性能（对应时间）和显存使用（对应空间）是非常重要的优化手段。
它让我们从**完成**跨越到**完美**。

文档作者：喂喂薇

在上一节利用 Nsight Systems 等工具对模型进行计算分析, 可以看到限制模型内存和计算资源的主要瓶颈主要还是在 Attention 注意力计算上。因此, 我们优化的目标也会集中在此。仔细分析过后可以发现, Attention 计算的瓶颈主要在于 IO 感知上, FlashAttention 的优化方法应运而生。

9.1. GPU 内存架构

参考资料 9.1

- <https://www.cnblogs.com/ArsenalFanInECNU/p/18021724>

9.1.1. 物理内存架构: 片上内存与片下内存

从硬件位置来看, GPU 内存主要分为两类: **片上 (on-chip) 内存** 和 **片下 (off-chip) 内存**。

- **片上内存 (On-chip Memory)**: 位于 GPU 芯片内部, 主要用作高速缓存 (Cache)、共享内存 (Shared Memory) 和寄存器 (Register)。

- **特点**: 速度极快, 但存储空间非常小。
- **硬件类型**: 通常是 **SRAM** (静态随机存取存储器), 其优点是访问速度快, 无需刷新即可保存数据。

- **片下内存 (Off-chip Memory)**: 位于 GPU 芯片外部, 主要用作全局内存 (Global Memory), 也就是我们常说的“显存”。

- **特点**: 容量大, 但速度相对较慢。
- **硬件类型**: 通常是 **HBM** (高带宽内存), 它通过将多个 DDR 芯片堆叠并与 GPU 封装在一起, 以实现大容量和高位宽。HBM 属于 **DRAM** (动态随机存取存储器), 其成本较低、密度高, 但访问速度慢于 SRAM。

9.1 GPU 内存架构 . . .	101
9.1.1 物理内存架构: 片上内存与片下内存	101
9.1.2 逻辑内存层次与功能划分	102
9.2 FlashAttention . . .	103
9.2.1 GPU 的 FLOPs 计算能力与内存吞吐能力	104
9.2.2 I/O 感知	104
9.2.3 FlashAttention 如何充分考虑 GPU 的内存层级结构?	104
9.2.4 实现细节	106
9.2.5 Flash Attention 的计算量分析	109
9.3 FlashAttention2 与 FlashAttention1 的区别	110
9.3.1 FlashAttentionv3	112
9.4 Triton	112
9.4.1 Triton 的核心思想: 并行化与分块 (Parallelism and Tiling)	113
9.4.2 Triton 的“分块”特性 . . .	114
9.4.3 Triton 和 C/C++ 的编译方式的联系和区别	115
9.5 用 Triton 优化 FlashAttention . . .	116
9.5.1 前向传播	116
9.5.2 反向传播	118

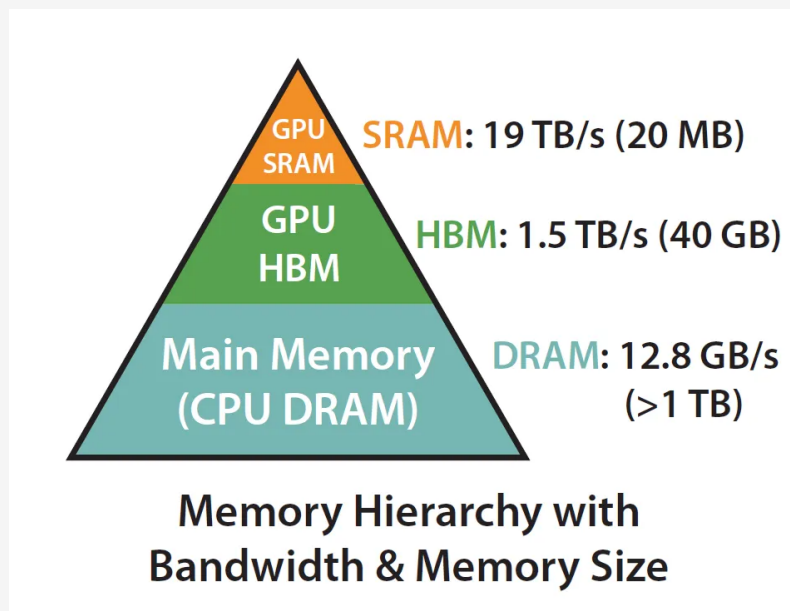


图 9.1: GPU 内存架构

9.1.2. 逻辑内存层次与功能划分

在 CUDA 编程模型中, GPU 内存根据其作用域、生命周期和访问特性被划分为不同的逻辑类型。其速度和容量关系通常是: **寄存器 > 共享内存/L1 缓存 > L2 缓存 > 全局内存**。

以下是各类内存的详细介绍:

1. 寄存器 (Register)

- **位置与速度**: 位于片上, 是 GPU 中速度最快的内存空间。
- **作用域**: 每个线程独享的私有资源, 用于存储线程内频繁使用的临时变量。
- **生命周期**: 与核函数 (Kernel) 的执行周期一致, 核函数运行结束即被释放。
- **特点**: 容量非常有限。如果一个线程需要的变量过多, 寄存器会发生“溢出”, 多出的变量将被存放到速度慢得多的本地内存中, 严重影响性能。

2. 本地内存 (Local Memory)

- **位置与速度**: 物理上它与全局内存位于同一区域, 即片下的 HBM/DRAM 中, 因此访问延迟高、速度慢。
- **作用域**: 和寄存器一样, 是每个线程的私有内存。
- **用途**: 主要用于存放两种数据: □ 编译器无法确定索引的本地数组; □ 因体积过大或数量过多而无法放入寄存器的变量 (即寄存器溢出)。

3. 共享内存 (Shared Memory)

- **位置与速度**: 位于片上 (on-chip), 是可编程的内存, 访问速度非常快, 几乎和寄存器一样。
- **作用域**: 被一个线程块 (Block) 内的所有线程共享, 可用于块内线程间的高效通信。不同线程块之间无法通过共享内存通信。
- **生命周期**: 与线程块的生命周期一致, 线程块执行开始时分配, 执行结束时释放。

4. 全局内存 (Global Memory)

- **位置与速度**: 位于片下 (off-chip) 的 DRAM/HBM 中, 是 GPU 上容量最大但访问速度最慢的内存空间。通过 ‘cudaMalloc’ 等函数分配的内存就在这里。
- **作用域**: 所有线程都可以访问, 是实现不同线程块之间数据通信的唯一途径。
- **生命周期**: 与整个应用程序的生命周期相同, 除非被显式释放。
- **缓存**: 对全局内存的访问会经过 L1 和 L2 缓存来提速。

5. 常量内存 (Constant Memory)

- **位置与速度**: 物理上驻留在片下的设备内存中, 但每个 SM 都有一个专用的只读常量缓存 (Constant Cache), 因此读取速度很快。
- **作用域**: 对所有线程可见, 但它是只读的。
- **用途**: 主要用于存储在核函数执行期间不会改变的数据。当一个 Warp (一组线程) 中的多个线程需要访问同一个常量数据时, 常量缓存可以实现广播 (broadcast), 一次性满足所有请求, 避免了串行访问, 从而提高效率。

6. L1/L2 缓存 (Cache)

- **位置**: L1 缓存位于每个 SM 内部, 被该 SM 内的 CUDA 核心共享。L2 缓存则被 GPU 上所有的 SM 共享。
- **硬件类型**: 它们都是片上 SRAM。
- **作用**: 由系统自动控制, 对程序员不完全透明。它们主要用于缓存对慢速内存 (如全局内存和本地内存) 的访问, 以减少访问延迟。FlashAttention 这类算法的核心思想就是尽可能利用 L1/L2 缓存来减少对 HBM 的读写。

9.2. FlashAttention

参考资料 9.2

- **参考**: [FlashAttention-猛猿-知乎](#)
- **flashAttention 动画演示**-[bilibili](#)

开宗明义: FlashAttention 的背景是在当前原始的 Attention 在 GPU 中, 存写速度是逊色于计算速度的, 也就是说存写相比于计算时拖了后腿的。为了达到最佳效率, 我们需要减少多次存写, 哪怕以稍微降低计算速度的代价。

9.2.1. GPU 的 FLOPs 计算能力与内存吞吐能力

多年来，GPU 的计算能力（FLOPS）的增长速度比增加内存吞吐量（TB/s）更快。

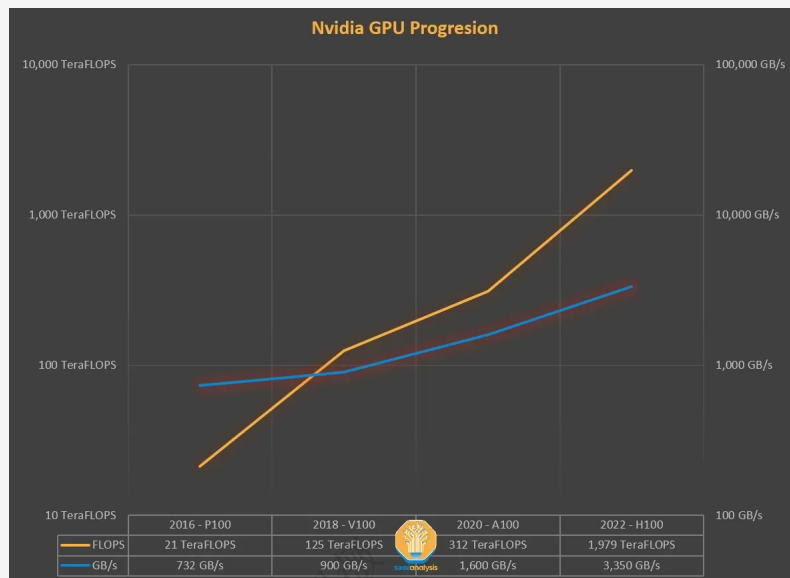


图 9.2: GPU 的 FLOPs 计算能力与内存吞吐能力

如果没有数据需要处理，那么额外的 FLOPS 的计算能力是没有意义的。总而言之，只有**存算**合理配合，才能达到最佳效率。

9.2.2. I/O 感知

IO 感知（IO-awareness）是一种在设计算法时充分考虑**硬件输入/输出（I/O）开销**的优化思想，特别是在 GPU 这类**计算速度远超内存访问速度**的设备上尤为重要。

在现代 GPU 中，性能瓶颈往往不是浮点运算（FLOPs）的速度，而是从**显存（HBM）读写数据的速度**。因此，IO 感知的算法旨在通过优化内存访问模式，特别是**减少对速度较慢、但容量较大的高带宽内存（HBM）的读写次数**，来减少整体的运行时钟时间（wall-clock time）。

简单来说，IO 感知的核心就是：**尽可能减少与慢速内存的数据交换，让计算尽可能在快速内存中完成。**

9.2.3. FlashAttention 如何充分考虑 GPU 的内存层级结构？

标准注意力算法的缺点

标准注意力算法没有感知到 IO 的成本，它会频繁地读写 HBM。例如，它需要计算并存储一个巨大的 $N \times N$ 的中间注意力矩阵（S 和 P）到 HBM 中，然后再从 HBM 中读出进行下一步计算。这些冗余的 HBM 读写操作（IO）占据了大量的计算时间，成为了性能瓶颈。

Transformer 的计算瓶颈不在运算能力，而在读写速度上。

Keynote 9.1

- **FLOPS**: 等同于 $FLOP/s$, 表示 **Floating Point Operations Per Second**, 即每秒执行的浮点数操作次数, 用于衡量硬件计算性能。
- **FLOPs**: 表示 **Floating Point Operations**, 表示某个算法的总计算量 (即总浮点运算次数), 用于衡量一个算法的复杂度。

FlashAttention 的性能提升方式

1. **利用 SRAM 进行分块计算 (Tiling)**: 为了避免对 HBM 的反复读写, FlashAttention 的核心思路是将多个操作融合成一个单一的 GPU 核 (Kernel), 并利用速度快 10 倍左右的 SRAM 进行计算。

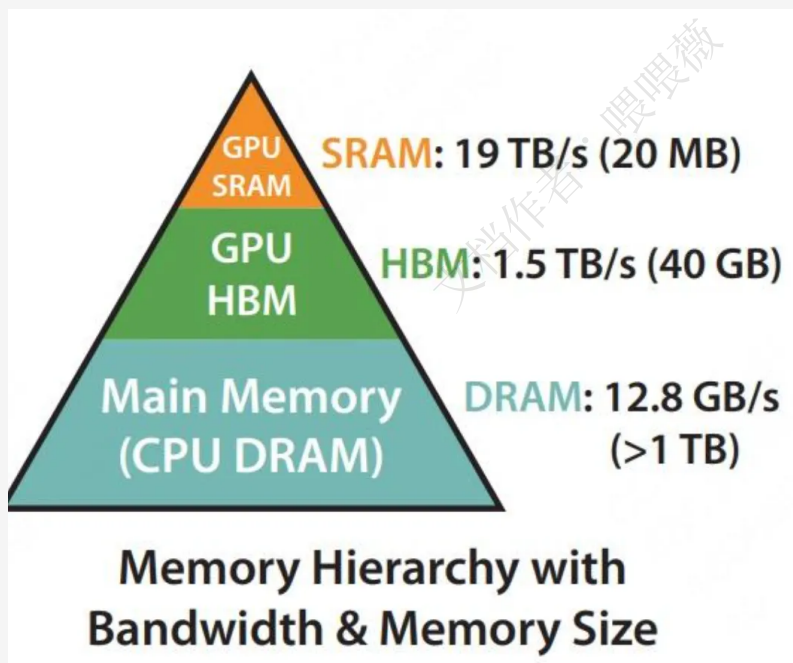


图 9.3: GPU 体系架构

- 它将输入的查询 (Q)、键 (K)、值 (V) 矩阵从 HBM 中切分成块 (Blocks/Tiles)。
- 然后, 它逐块将这些数据加载到高速的 SRAM 中。所有的中间计算步骤, 包括矩阵乘法和 Softmax, 都在 SRAM 内部完成, 完全不产生将巨大的 'N×N' 中间矩阵写回 HBM 的操作。
- 最后, 只将计算完成的最终输出结果从 SRAM 一次性写回 HBM。

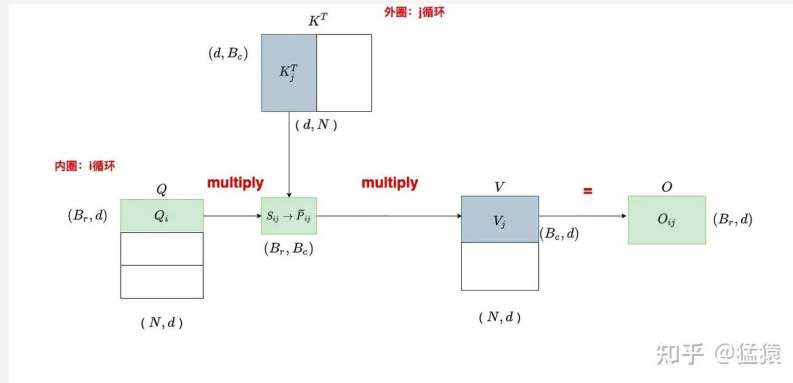


图 9.4: FlashAttention v1 内外循环

2. **通过重计算 (Recomputation) 减少内存占用**: 为了进一步优化内存, FlashAttention 在前向传播时不会保存用于反向传播的中间注意力矩阵 (S 和 P)。在反向传播时, 它会利用保存在 SRAM 上的输入块以及少量统计数据, 快速地重新计算出这些中间值。这种“以时间换空间”的策略极大地节省了 HBM 的占用。

9.2.4. 实现细节

原始 Attention 计算

$$\text{Attention}(Q, K, V) = \text{softmax}((Q * K^T) / d_k) * V$$

缩放部分 $\sqrt{d_k}$ 的计算不占额外存储, 可忽略掉。主要考量 $Q * K^T \text{softmax}() * V$ 这三个运算

FlashAttention 的分块计算流程

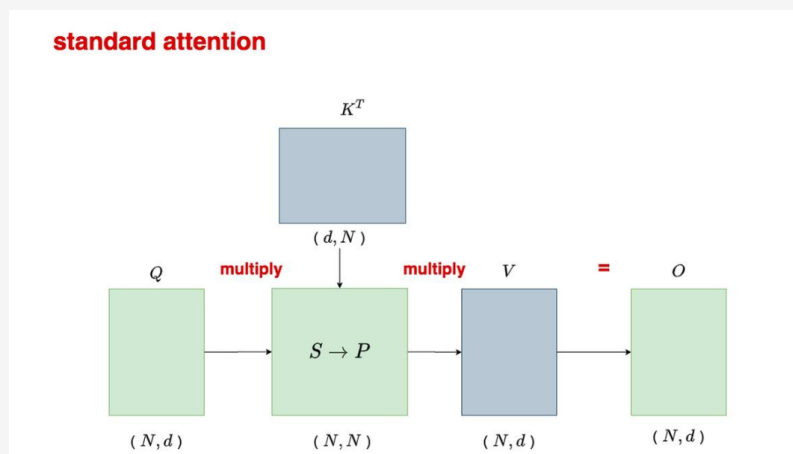


图 9.5: 分块计算

1. 首先, 将 Q 矩阵 (形状为 (N, d)) 按照“行”切为 T_r 块 (block), 每块的长度为 B_r 。用 Q_i 来表示切完后的某块矩阵, 则 Q_i 的维度为 (B_r, d) 。不难

理解， Q_i 中存储着某 B_r 个 token 的 query 信息。

2. 然后，将 K^T 矩阵(形状为 (d, N))按照“列”切为 T_c 块，每块的长度为 B_c 。用 K_j^T 表示切完后的某块矩阵，则 K_j^T 的维度为 (d, B_c) 。易知 K_j^T 中存储着某 B_c 个 token 的 key 信息。
3. 同样，将 V 矩阵也按照“行”切为 T_c 块，每块长度为 B_c 。用 V_j 表示切完后的某块矩阵，则 V_j 的维度为 (B_c, d) 。易知 V_j 中存储着某 B_c 个 token 的 value 信息。

计算初始 attention 分数：

$$S_{ij} = Q_i * K_j^T = (B_r, d) * (d, B_c) = (B_r, B_c)$$

图中的 S_{ij} 表示前 B_r 个 token 和前 B_c 个 token 间的原始相关性分数。

具体循环中的操作示意：

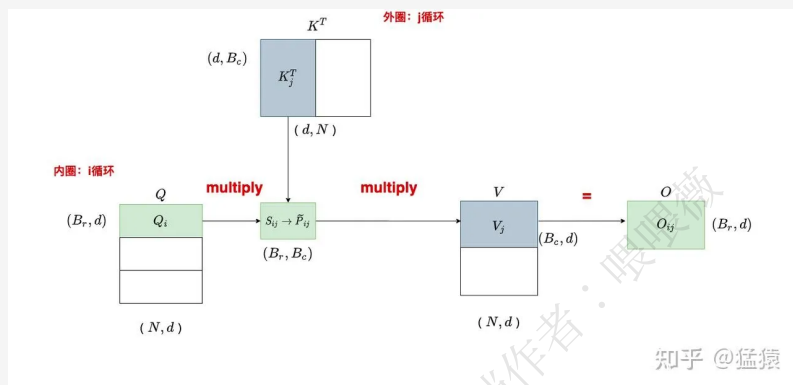


图 9.6: FlashAttention v1 内外循环

softmax 计算操作

原始 softmax 计算： $\text{softmax}(x_i) = \exp(x_i - m) / \sum_j \exp(x_j - m)$ 其中 m 是某一行列的最大值，减去它是为了数值稳定性。

定义：

- $m(x)$ ：标准场景下，该行的全局最大值
- $m(x^{(1)})$ ：分块 1 的全局最大值
- $m(x^{(2)})$ ：分块 2 的全局最大值

那么易知： $m(x) = m([x^{(1)}, x^{(2)}]) = \max(m(x^{(1)}), m(x^{(2)}))$

- $f(x)$ ：标准场景下， $e^{x-m(x)}$ 的结果
- $f(x^{(1)})$ ：分块场景下， $e^{x^{(1)}-m(x^{(1)})}$ 的结果
- $f(x^{(2)})$ ：分块场景下， $e^{x^{(2)}-m(x^{(2)})}$ 的结果

那么易知： $f(x) = [e^{m(x^{(1)})-m(x)} f(x^{(1)}), e^{m(x^{(2)})-m(x)} f(x^{(2)})]$

- $l(x)$ ：标准场景下， $\text{rowsum}[f(x)]$ 的结果

● $l(x^{(1)})$: 分块场景下, $\text{rowsum}[f(x^{(1)})]$ 的结果

● $l(x^{(2)})$: 分块场景下, $\text{rowsum}[f(x^{(2)})]$ 的结果

那么易知: $l(x) = [l(x^{(1)}), l(x^{(2)})]$ 。

$$\text{softmax}(x) = \frac{f(x)}{l(x)} = \frac{[e^{m(x^{(1)})-m(x)} f(x^{(1)}), e^{m(x^{(2)})-m(x)} f(x^{(2)})]}{e^{m(x^{(1)})-m(x)} l(x^{(1)}) + e^{m(x^{(2)})-m(x)} l(x^{(2)})}$$

分块计算操作:

$$m(x) = \max_i (x_i)$$

$$f(x) = [e^{x_1-m(x)}, \dots, e^{x_n-m(x)}]$$

$$l(x) = \sum_i f(x)_i$$

$$\text{softmax}(x) = \frac{f(x)}{l(x)}$$

FlashAttention 的在线 Softmax 算法

FlashAttention 的核心思想是: 我们可以**迭代地**更新 Softmax 的计算结果。每当一个新的数据块到来时, 我们用它来**修正**之前基于旧数据块计算出的 (不完整的) 结果。

让我们把注意力计算中的一行 (例如, Q_i 和所有 K_j 的点积) 看作是要计算 Softmax 的输入向量 x 。这个向量 x 被分成了多个块 $x_1, x_2, \dots, x_{T_c}$ 。

算法为这一行维护三个核心变量:

- O : 当前的输出 (对 V 的加权和)。
- m : 到目前为止见过的所有 x 元素中的最大值 (**running maximum**)。
- l : 到目前为止 Softmax 分母的累加和 (**running denominator**)。

现在, 我们模拟算法的迭代过程:

假设我们已经处理了第 1 到 $j-1$ 个块, 得到了当前的统计量 m_{old} 和 l_{old} , 以及当前的输出 O_{old} 。

现在, 我们处理第 j 个块, x_j 。

1. 计算当前块的局部统计量

我们只看当前块 x_j , 可以计算出它的:

- 局部最大值: $m_j = \max(x_j)$
- 局部 Softmax 分子: $P_{tilde_j} = e^{x_j - m_j}$ (注意, 减去的是局部最大值)
- 局部 Softmax 分母: $l_j = \text{sum}(P_{tilde_j})$

2. 合并统计量, 更新全局最大

新的全局最大值, 一定是旧的全局最大值和当前块最大值中的较大者: $m_{new} = \max(m_{old}, m_j)$

3. 重新缩放 (Re-scaling) 并更新分母 l

这是最关键的数学技巧。我们之前的 l_{old} 是基于 m_{old} 计算的, 而 l_j 是基于 m_j 计算的。现在有了新的全局最大值 m_{new} , 我们需要把这两部分都统一到新的基准上再相加。

$$l_{new} = l_{old} * e^{m_{old}-m_{new}} + l_j * e^{m_j-m_{new}}$$

理解这个公式:

- l_{old} 最初是 $e^{x_{old}-m_{old}}$ 。乘以 $e^{m_{old}-m_{new}}$ 后，它就变成了 $e^{x_{old}-m_{new}}$ 。
- l_j 最初是 $e^{x_j-m_j}$ 。乘以 $e^{m_j-m_{new}}$ 后，它就变成了 $e^{x_j-m_{new}}$ 。
- 两者相加，就得到了处理完 ‘j’ 块数据后，基于新的全局最大值 m_{new} 的正确的分母总和！

4. 重新缩放并更新输出 O

输出 O 是对值矩阵 V 的加权和。它的更新逻辑和分母 l 完全一样，也需要重新缩放。

$$O_{new} = (O_{old} * l_{old} * e^{(m_{old} - m_{new})} + (P_{tilde_j} * V_j) * e^{(m_j - m_{new})}) / l_{new}$$

理解这个公式:

- 分子部分 (...) 是计算了未归一化的加权和，并将其统一到了 m_{new} 基准上。
- $P_{tilde_j} * V_j$ 是当前块 x_j 对 V_j 的加权和。
- 最后，除以新的、正确的总分母 l_{new} ，就得到了更新后的、正确的输出 O 。

在论文的算法图中，这个公式被写作： $O_i \text{diag}(l_i^{new})^{-1} (\text{diag}(l_i) * e^{(m_i - m_i^{new})} * O_i + e^{(m_i - m_i^{new})} * P - \tilde{t}_{ij} * V_j)$

这只是上述逻辑的矩阵化写法。 $\text{diag}(l)$ 乘以 O 是为了还原出未归一化的加权和。

在反向传播中，因为中间计算的结果没有存储，所以要重新计算一下。

9.2.5. Flash Attention 的计算量分析

主要设计两部分矩阵计算：

对于 $S_{ij} = Q_i K_j^T$ ，其中 $Q_i \in \mathbb{R}^{B_r \times d}$, $K_j^T \in \mathbb{R}^{d \times B_c}$ 。根据前置知识，求 S_{ij} 的计算量为 $O(B_r B_c d)$ 。

对于 $\tilde{P}_{ij} V_j$ ，其中 $\tilde{P}_{ij} \in \mathbb{R}^{B_r \times B_c}$, $V_j \in \mathbb{R}^{B_c \times d}$ 。则这里的计算量同样为 $O(B_r B_c d)$ 。

接下来我们看一共计算了多少次 (1) 和 (2)，也就是执行了多少次内循环：

$$T_c T_r = \frac{N}{B_c} \frac{N}{B_r}$$

综合以上三点，flash attention 的 forward 计算量为： $O\left(\frac{N^2}{B_c B_r} B_r B_c d\right) = O(N^2 d)$

注意，因为计算量是用大 O 阶表示的，所以这里我们把常数项都省略了。

IO 复杂度分析

操作 (Operation)	标准注意力 (Standard Attention)	FlashAttention	备注 (Remarks)
HBM -> SRAM (读操作)			
读 Q, K, V	$O(Nd)$	$O(Nd)$	这是无法避免的初始加载。
读中间矩阵 S 或 P ($N \times N$)	$O(N^2)$	$\emptyset$ (不读取)	核心区别: 标准注意力需 S/P 写回 HBM 再读出, FlashAttention 完全避免这一步。
SRAM -> HBM (写操作)			
写中间矩阵 S 或 P ($N \times N$)	$O(N^2)$	$\emptyset$ (不写入)	核心区别: FlashAttention 中间计算结果保留在 SRAM 中, 用完即弃。
写最终输出 O	$O(Nd)$	$O(Nd)$	最终结果必须写回。
总 I/O 复杂度	$O(N^2 + Nd)$	$O(Nd)$	当 $N \gg d$ 时, N^2 项成为不可逾越的瓶颈。

9.3. FlashAttention2 与 FlashAttention1 的区别

v2 主要提升有两点:

1. 引入多线程提升了并行的能力
2. 继续优化了细节, 比如延迟归一化使得内部的除法运算进一步减少。

v2 最主要的改进: 原 v1 在进行 QK^TV 的分块计算时, 是把 K^T 放到最外层循环的, QV 放到内层循环。v2 将 Q 放到最外层, K^TV 放到了内层。

从图里面可以看到, 对于 v1, K^T 是外层循环, 这时候每次和 Q 相乘的内循环得到的中间结果是红色圈中的“列”, 这个列是没办法直接和 V 相乘得到完整的结果的, 它需要等到下一次 K^T 的循环得到下一个列与 V 相乘得到的结果求和才是最终的 O 。

而对于 v2, Q 是外层循环, Q 的每一行是每次循环的变量, 这时候这一行可以和所有的内循环的 K^T 列相乘得到中间结果的行, 即图中画蓝、红、紫的圈, 这几个行可以单独与 V 也进行相乘, 得到最终结果 O_{00} 、 O_{10} 等单个元素, 不需要等待下一次循环。这样做好处就是可以把 Q 按照行进行各自独立的分割, 每一行之间互不影响, 就可以放到多个线程里面并行优化。

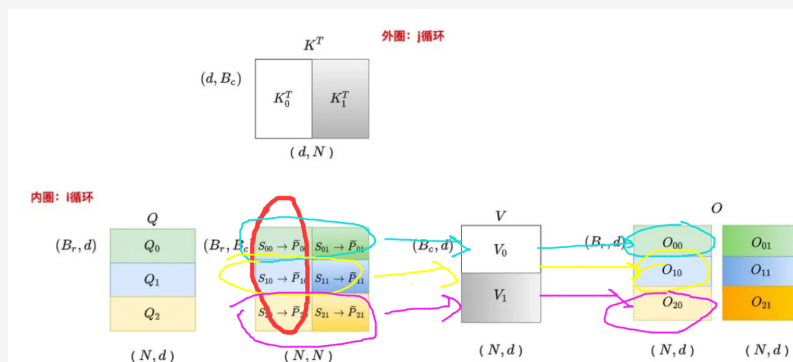


图 9.7: FlashAttention v2 内外循环

并行策略的进一步增强

- **FlashAttention-1**: 其并行化主要在 **批次 (batch size)** 和 **头 (head)** 这两个维度上进行。每个注意力头分配一个线程块。当序列很长而批次和头数较少时，可并行的线程块数量可能远少于 GPU 上的流多处理器 (SM) 数量，导致 GPU 利用率低下。
- **FlashAttention-2**: 在 V1 的基础上，增加了在 **序列长度 (sequence length)** 维度上的并行化。
 - **前向传播**: 将 Q 矩阵按行切分，不同的 **行块 (row blocks)** 分配给不同的线程块并行计算。
 - **反向传播**: 由于梯度计算的依赖关系不同 (dK 和 dV 需要按行累加)，为了优化，反向传播是按 **列块 (column blocks)** 分配给不同的线程块并行计算的。

Warp 间工作分区的优化：减少内部通信

即使在单个线程块 (Thread Block) 内部，V2 也优化了工作分配方式。一个线程块由多个 Warp (通常每个包含 32 个线程) 组成。

- **FlashAttention-1**: 采用了一种称为 **“Split-K”** 的策略。它将 K 和 V 矩阵沿着序列维度切分给不同的 Warp，而 Q 块对所有 Warp 可见。这种方式的缺点是，每个 Warp 计算出的是部分结果，为了得到最终的行输出，必须将这些中间结果写入共享内存 (SRAM)，进行 Warp 间的同步和聚合 (相加)，这引入了不必要的通信和共享内存读写开销，成为性能瓶颈。
- **FlashAttention-2**: 转而采用 **“Split-Q”** 策略。它将 Q 矩阵切分给不同的 Warp，而 K 和 V 块对所有 Warp 可见。因为不同 Q 的计算是独立的，每个 Warp 在计算完自己的 $(Q_{slice} * K^T) * V$ 后，直接得到最终输出 O 的一个分片，**无需与其他 Warp 进行通信或聚合**。这极大地减少了共享内存的读写和同步开销，是 V2 速度提升的关键之一。

算法层面的优化

FlashAttention-2 在算法层面进行了一些精简，以减少开销较高的非矩阵乘法浮点运算（non-matmul FLOPs）。

- **延迟归一化 (Rescaling)**: 在 V1 中，每一次内循环迭代，都需要用当前的局部 softmax 分母 l 来重新缩放（rescale）累积的输出 O ，这涉及到除法运算。FlashAttention-2 优化了这一点，在内循环中，它只更新 O 的“未归一化”版本，将最终的除法操作**延迟到外循环结束时才执行一次**。这个调整减少了中间步骤的除法运算次数，更充分地利用了为矩阵乘法优化的 Tensor Cores。
- **反向传播存储优化**: V1 为了反向传播时的重计算，需要从 HBM 中读取并保存两个统计量 m (最大值) 和 l (exp 分数总和)。V2 则将它们**合并，只存储一个 $L = m + \log(l)$ (即 log-sum-exp)**，在反向传播时由此恢复所需信息。这进一步减少了一次 HBM 的 I/O 操作，节省了内存带宽。

9.3.1. FlashAttentionv3

相比于 FlashAttentionv1、v2，v3 没有太多算法上的创新，主要有两个性能提升的点：

1. **矩阵计算 (GEMM) 和 softmax 计算流水线化**，交错并行，不严格顺序执行，而是会有时间的重叠。
2. **硬件加速的 FP8 低精度计算与精度保持**。

9.4. Triton

参考资料 9.3

- Triton 官方中文文档: <https://triton-lang.cn/main/index.html>

Triton 是一个由 OpenAI 开发的、基于 **Python** 的编程语言的编译器，可以让我们用类似 Python 的语法为 GPU 编写高效的**计算内核 (Kernel)**。这样就不用学习复杂的 CUDA C++ 编程，就能写出优化 CUDA 代码性能的**自定义算子**，降低了 GPU 编程的门槛。

我们以实现 Triton 的向量加法为例子：

假设我们想实现一个很简单的“向量 + 向量 $\rightarrow$ 向量”的加法计算。绝大多数情况我们都不会考虑底层优化，而是直接在 PyTorch 上直接 **`c = a + b`** 了，这样做当然可以，而且 Pytorch 也会有它自己的底层优化机制。但如果你想做一个 PyTorch 没有的、非标准的、或者多个操作想融合在一起的操作（比如 **`c = a * b + sin(a)`**），PyTorch 就需要依次调用多个内核，达不到最佳性能。

这个时候如果我们还想实现最佳的性能，方式就是写一个最佳的优化算子，一套“组合拳”直接实现 **`a * b + sin(a)`** 计算。传统的方式就是手动写 CUDA C++，直接与 GPU 底层交互，不过需要我们自己管理线程块（Thread Blocks）、线程（Threads）、共享内存（Shared Memory）、内存对齐和合并访问（Coalescing）等一系列复杂的底层细节。

Extension 9.1 算子融合

算子融合是将多个独立的小操作（如乘法、加法、Mask 操作）合并成一个单一的、更高效的“大”操作（称为 ****Kernel****）。这大大减少了从内存中读取和写入数据的次数，也减少了启动 GPU 计算任务的开销。

这个时候 Triton 的作用就得以体现，它让我们用类似 Python 的较为简单的语法来实现底层 CUDA 优化的性能。

9.4.1. Triton 的核心思想: 并行化与分块 (Parallelism and Tiling)

Triton 编程的核心是**并行化**。我们不会写一个程序来处理整个矩阵‘**X**’，而是会写一个**程序模板（Triton Kernel）**，GPU 会同时启动成百上千个这个程序的**实例（Program Instance）**，每个实例只负责一小部分工作。

还是以我们之前说的向量加法为例子：

```
import torch
import triton
import triton.language as tl

@triton.jit # Triton 内核函数装饰器，只有装饰器内的函数才会被 Triton 编译
def add_kernel(
    x_ptr, # 指向输入向量 a 的指针，就和 C 语言的指针差不多，直接和显存地址交互
    y_ptr, # 指向输入向量 b 的指针
    output_ptr, # 指向输出向量 c 的指针
    n_elements, # 向量中的元素总数
    BLOCK_SIZE: tl.constexpr, # 块大小，一个编译时常量
):
    # --- 内核的执行逻辑 ---
    # GPU 通过大规模并行来获得高性能。Triton 会启动很多个 add_kernel 的程序实例来同
    # 时处理数据。pid 就是当前这个实例的唯一编号（从 0 开始）。
    # 计算当前这个程序实例的 ID
    pid = tl.program_id(axis=0)
    # 计算当前程序实例要处理的数据块的偏移量
    # tl.arange 生成一个 [0, 1, 2, ..., BLOCK_SIZE-1] 的数组
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE) # 偏移量 它定义了当前这个 pid
    # 对应的程序实例负责处理哪一段数据。

    # 创建一个 mask，防止内存越界
    # 比如向量长度是 1000，BLOCK_SIZE 是 1024，最后一个块会访问到不存在的元素
    mask = offsets < n_elements

    # 从全局内存（HBM）加载数据块
    # mask=mask 保证了只加载有效的数据
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)

    # 执行计算
    output = x + y

    # 将计算结果写回全局内存
```

```
tl.store(output_ptr + offsets, output, mask=mask)
```

接下来我们将在 CPU 上调用这段函数

```
size = 98432 # 创建输入数据
a = torch.randn((size,), device='cuda')
b = torch.randn((size,), device='cuda')
output = torch.empty_like(a) # 准备一个空的输出 tensor
# 定义执行配置
BLOCK_SIZE = 1024
# grid 这个元组定义了我们要启动多少个 add_kernel 的程序实例。
grid = (triton.cdiv(size, BLOCK_SIZE),) # cdiv 是向上取整的除法
# 方括号里传入 grid 配置 像调用普通函数一样传入参数
add_kernel[grid](a, b, output, size, BLOCK_SIZE=BLOCK_SIZE)
# 验证结果
print(torch.allclose(output, a + b)) # 输出 True
```

总结下来有如下几个要点：

- Triton 内核函数一定要加 `@triton.jit` 修饰符
- 每个内核函数在被实际调用的时候都只是众多实例中的一个，它只负责部分计算，所以要确保它的计算在正确的地址上（对应**输入的指针 + 单位偏移量 offset**）
- 有了指针地址后才能从 GPU 上加载到数据，执行完我们想要的运算之后在写回去。
- 一些细节比如**mask**是考虑到了数据块的大小和真实输入的大小。

9.4.2. Triton 的”分块”特性

其实 Triton 的分块性能更好，刚才向量加法的例子没能体现，接下来我们讲解讲义上矩阵乘法的例子来充分证明这一点。

对于矩阵乘法 `C = A @ B`，我们不直接计算 C，而是把 C**分块 (tiling)**，我们利用 Triton 启动很多个实例，每个实例就去计算其中的一个小块，对于每一个小块的计算，执行以下步骤：

- 从输入矩阵 ‘A’ 中加载一**行块**，从矩阵 ‘B’ 中加载一**列块**。
- 执行 ‘`tl.dot`’（点积）运算，得到一个中间结果。
- 将 ‘A’ 的指针向右移动一个块的距离，‘B’ 的指针向下移动一个块的距离，加载下一对块。
- 执行 ‘`tl.dot`’ 并将结果**累加**到上一步的结果上。
- 重复 c, d 步，直到遍历完 ‘A’ 的所有列和 ‘B’ 的所有行。

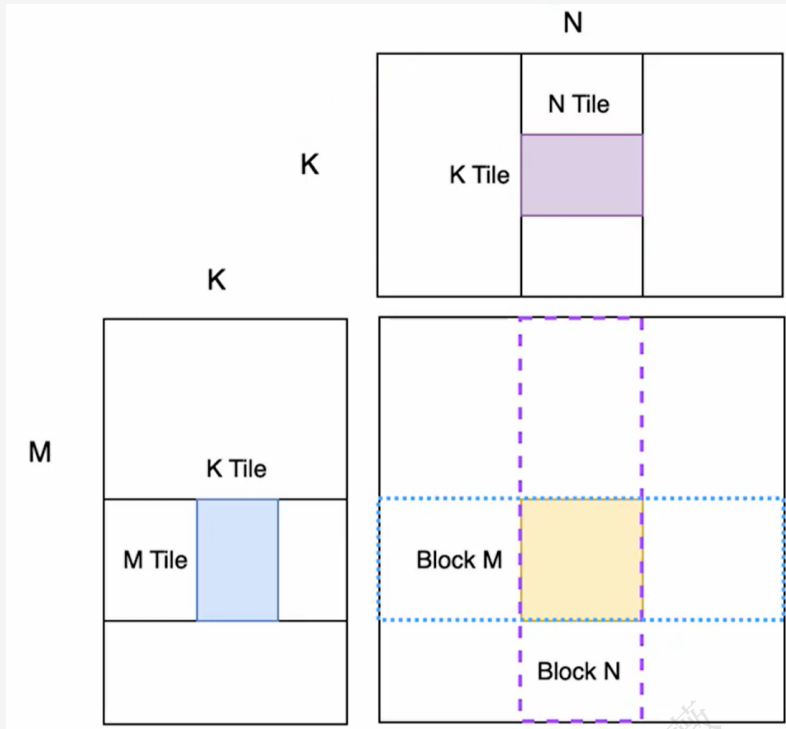


图 9.8: Triton 矩阵乘法

除此之外，我们会使用 `tl.make_block_ptr()` 这一智能指针来替代我们之前的手写的方式

```
a_block_ptr = tl.make_block_ptr(    # 创建一个描述 A 矩阵上数据块的“智能指针”
    base=a_ptr,                    # 矩阵的基地址
    shape=(M, K),                  # 整个矩阵的形状
    strides=(stride_am, stride_ak), # 矩阵的步长
    offsets=(pid_m * BLOCK_SIZE_M, 0), # 当前块的起始偏移（行，列）
    block_shape=(BLOCK_SIZE_M, BLOCK_SIZE_K), # 定义要操作的块形状
    order=(1, 0)                   # 内存布局顺序，通常是（1，0），代表行主序，即先行
    ↪ 后列，（0,1）就反过来
)

# 在循环中，使用 tl.advance 来移动指针
a_block_ptr = tl.advance(a_block_ptr, (0, BLOCK_SIZE_K))
# 语义非常清晰：“在第 1 个维度（K 维）上前进 BLOCK_SIZE_K”

# 加载时，使用 boundary_check 自动处理边界，不用 mask 了
a = tl.load(a_block_ptr, boundary_check=(0, 1)) # 检查第 0 和第 1 个维度
```

9.4.3. Triton 和 C/C++ 的编译方式的区别和联系

C/C++ 编译器比较通用，负责将通用程序翻译成 CPU 或 GPU 代码，开发者需要手动处理大量底层细节。而 Triton 编译器**垂直性更强**，它专门针对 GPU 高性能计算，接收高度抽象的、以**数据块**为单位的指令，并**自动**完成最困难的性能优化，将开发者从复杂的 GPU 硬件细节中解放出来。

C/C++ (含 CUDA C++): 编程模型更底层。在为 GPU 编写 CUDA C++ 时，你需要手动管理：

- **线程层级**: 精确定义线程块（Block）和网格（Grid）的维度。
- **内存管理**: 手动管理共享内存的分配和同步，以实现线程间通信和数据重用。
- **内存访问**: 程序员需要自己精心设计内存访问模式，以确保“合并访问”，避免性能大幅下降。

Triton: 编程模型抽象层次更高。你不再直接操作单个线程，而是从**程序实例**的**视角**出发，操作**数据块**。

- **块级编程**: 你通过 ‘tl.program_id’ 获取当前程序实例的 ID，然后加载、计算和存储一整块数据（例如 ‘tl.load(ptr + offsets)’）。
- **自动优化**: 你无需关心共享内存、合并访问或线程同步。Triton 编译器会分析你的块级操作，**自动生成**管理共享内存、保证合并访问的底层 PTX 代码。它为你处理了最复杂的部分。

9.5. 用 Triton 优化 FlashAttention

9.5.1. 前向传播

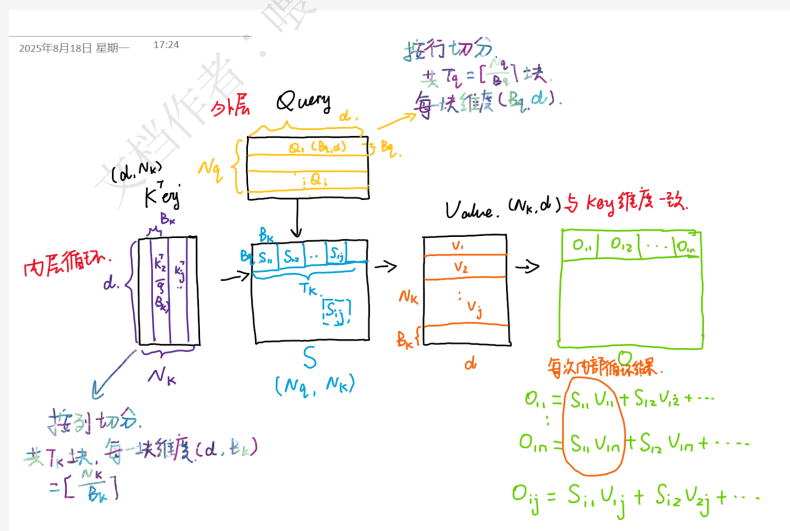


图 9.9: FlashAttention 计算示意图

就像我们之前提到的，我们想要实现性能的提高，就要尽可能地**减少对 HBM 的读写**，因为计算速度远大于通信速度。这也是我们利用 Triton 优化 FlashAttention 的核心思想。

在讲义中，我们 Triton 被要求设置的网格维度是 $(T_q, \text{batch_size})$ ，也就是并行的两个维度。事实上，Triton 可以并行的数量不止是两个维度，并且一般 FlashAttention 并行的维度是三个，分别是 $(T_q, \text{batch_size}, \text{head})$ 。讲义为了简化，只要求并行两个维度，取消了多头。

Extension 9.2 为什么在 FlashAttention 中 Triton 除了批次和多头之外只并行了 Q?

还是我们之前所说的，我们的核心思想是为了尽可能减少通信，用计算代替通信。如果把 K 也并行起来，那么每次一个小线程就是计算一小块 Q 和一小块 K 的点积，而最后要得到 softmax 的结果，需要一行所有的点积和，这就需要每个线程块之间进行通信，这就会大大降低性能。只并行 Q，那么一个线程就可以得到 S 一行的所有值，就不必再通信了。

在每一个线程的前向传播中， K^T 的小块是它的列， V 的小块是它的行。我们中间会得到 $S = \frac{QK^T}{\sqrt{d_k}}$ 的小块，用 M 来维护 S 每一行的最大值， L 来维护 S 每一行的 exp 分数总和 (随着循环计算的进行， M 和 L 会不断更新)。然后我们用 $P = \exp(S - M)$ 的小块来得到 softmax 的结果，用 $O = PV$ 的小块来得到输出。

薇言大义 9.1 这里我觉得很值得仔细想清楚的一点是后面的 O 矩阵的计算，它不是想前面 S 那样，每次一个线程就得到一块 S_{ij} ，而是每次一个线程就得到一整行 O_i ，然后随着循环不断叠加更新，最后得到最终的 O_i 。

```
Q_i = tl.load(Q_block_ptr) #(Q_TILE_SIZE, D)
O_i_acc = tl.zeros((Q_TILE_SIZE, D), dtype=tl.float32) #(Q_TILE_SIZE, D)
L_i_acc = tl.zeros((Q_TILE_SIZE, 1), dtype=tl.float32) #(Q_TILE_SIZE, 1)
M_i_acc = tl.full((Q_TILE_SIZE, 1), float('-inf'), dtype=tl.float32) #(Q_TILE_SIZE,
↪ 1)
# 外层循环：对 K/V 的序列长度进行分块
for j in range(tl.cdiv(N_KEYS, K_TILE_SIZE)):
    K_j = tl.load(K_block_ptr) #(K_TILE_SIZE, D)
    V_j = tl.load(V_block_ptr) #(K_TILE_SIZE, D)

    S_ij = tl.dot(Q_i, K_j.T) * scale #(Q_TILE_SIZE, K_TILE_SIZE)
    M_i_new = tl.max(S_ij, axis=1, keep_dims=True) #(Q_TILE_SIZE, 1)
    P_ij = tl.exp(S_ij - M_i_new) #(Q_TILE_SIZE, K_TILE_SIZE)
    L_i_new = tl.exp(M_i_acc - M_i_new) * L_i_acc + tl.sum(P_ij, axis=1,
↪ keep_dims=True) #(Q_TILE_SIZE, 1)
    # 数据对齐
    P_ij_cast = P_ij.to(V_block_ptr.type.element_ty) # 这代表从指针全局内存中读取类型，
↪ 用 V_j.dtype 理论上也行。
    O_i_new = tl.exp(M_i_acc - M_i_new) * O_i_acc + tl.dot(P_ij_cast, V_j)
    ↪ #(Q_TILE_SIZE, D)
    # 与 pytorch 代码不同，这里必须显式地更新 M_i_acc 和 O_i_acc，因为下一个循环不会记住
    ↪ 上一次的值
    M_i_acc = M_i_new
    O_i_acc = O_i_new
    L_i_acc = L_i_new

# 只有 K 和 V 指针在 K 维度上前进，遍历所有 K/V 分块
K_block_ptr = K_block_ptr.advance((K_TILE_SIZE, 0))
V_block_ptr = V_block_ptr.advance((K_TILE_SIZE, 0))
```

```

O_i = O_i_acc / L_i_acc # 全局归一化
L_i = M_i_acc + tl.log(L_i_acc) # 这个保存下来给反向传播用
tl.store(O_block_ptr, O_i)
tl.store(L_block_ptr, L_i)

```

9.5.2. 反向传播

反向传播计算公式:

$$S = \frac{QK^T}{\sqrt{d}}$$

$$P_{ij} = \exp(S_{ij} - L_i)$$

$$dV = P^T dO$$

$$dP = dO V^T$$

$$dS_{ij} = P_{ii} \circ (dP_{ii} - D_i)$$

$$dQ = \frac{dSK}{\sqrt{d}}$$

$$dK = \frac{dS^T Q}{\sqrt{d}}$$

正向计算公式:

$$S = \frac{QK^T}{\sqrt{d}}$$

$$P_{ij} = \text{softmax}_j(S)_{ij}$$

$$O = PV$$

反向传播:

根据链式法则: 若 $Y = AX$, 则 $\frac{\partial L}{\partial X} = A^T \frac{\partial L}{\partial Y}$

则 V 的梯度 dV (也就是 $\frac{\partial L}{\partial V}$ 损失)

$$\text{有 } dV = P^T dO$$

$$\text{而 } dP = V dO^T \Rightarrow dP = dO V^T$$

$$\Rightarrow \text{同理可求 } dQ = \frac{dSK}{\sqrt{d}}, dK = \frac{dS^T Q}{\sqrt{d}}$$

★ 对于 $P_{ij} = \text{softmax}_j(S)_{ij}$ 这一步的反向梯度推导:

$$\text{元素级: } P_j = \frac{e^{S_j}}{\sum_k e^{S_k}}$$

分母 = Z

① 若 j 恰好与 k 相等

$$\text{则 } \frac{\partial P_j}{\partial S_k} = \frac{e^{S_j} Z - e^{S_j} \cdot e^{S_j}}{Z^2} = \frac{e^{S_j}}{Z} (1 - \frac{e^{S_j}}{Z}) = P_j (1 - P_j)$$

② 若 $j \neq k$.

$$\text{则 } \frac{\partial P_j}{\partial S_k} = \frac{0 - e^{S_j} \cdot e^{S_k}}{Z^2} = -P_j P_k$$

换算到矩阵中, 只有 $j=k$ 也就是行列一致对角的时候值为 $P_j (1 - P_j)$.

其余为 $-P_j P_k$.

$$\Rightarrow \text{diag}(P) - PP^T$$

$$\therefore \text{原式 } dS_i = d - \text{softmax}(dP_i) = [\text{diag}(P_i) - P_i P_i^T] d P_i$$

图 9.10: 反向传播数学公式推导

反向传播并不会直接利用前向传播的所有计算结果，而是需要**重新计算**，同样还是为了尽可能减少通信，用计算代替通信。

反向传播也要像前向传播那样按照循环来写，只是会更容易，因为参数里面会给 Q, K, V, O, L ，所以总体上只需要按照公式来

反向传播的逻辑（链式法则）：根据链式法则，一个变量的梯度等于所有流经它的路径上的梯度之和。

对于 dK 。 K 的第 j 个分块 K_j ，在前向传播中，它不仅和 Q 的第 0 个分块 Q_0 作用了，也和 $Q_1, Q_2, \dots, Q_{Tq-1}$ 都发生了作用。

当我们的外层循环 $i = 0$ 时，我们计算了 K_j 因为和 Q_0 相互作用而产生的梯度贡献。当外层循环 $i = 1$ 时，我们又计算了 K_j 因为和 Q_1 相互作用而产生的梯度贡献。... 以此类推。

所以， K_j 的**总梯度**，必须是它与**所有** Q_i 相互作用产生的梯度的**总和**。这就是为什么用 $+=$ 来进行累加。如果用 $=$ 直接赋值，那么当外层循环 i 走到下一个值时，上一次计算出的梯度贡献就会被覆盖和丢失，最后得到的 dK 是不完整的，只反映了 K 和最后一个 Q 分块 Q_{Tq-1} 交互的梯度。

dV 的计算也是完全相同的道理。

对于 dQ ，逻辑是类似的，只是循环的内外层反了过来。在计算 dQ 的第 i 个分块 dQ_i 时，它也需要累加来自所有 K_j 和 V_j 的梯度贡献，所以在我们的实现中， $dQ[:, i * Bq : (i + 1) * Bq, :] += dQ_{ij}$ 也是在做累加。

与前向传播不同，反向传播是**把 K 放在了外层循环，把 Q 放在了内层循环**。这是为了让每一个并行的计算单元（线程块）都能独立、无冲突地完成一整块梯度（ dK 或 dV ）的计算，并将累加过程完全限制在各自高速的 SRAM 中。不过就算反过来也可以，只是会增加时延。

Extension 9.3 只从公式里面看，好像 dQ 和 dK 不都分别依赖了所有 K 和 Q 吗？那凭什么要把 K 放在外层循环，把 Q 放在内层循环？

这是一处非常重要的细节，把 K 放在外层是很重要的操作，这样做是因为如果把 Q 放在外层，需要很多线程块之间进行通信（下面会具体展开），而把 K 放在外层，就可以巧妙避免。

1. 已知: Q, K, V, O, L, dO ; 还可先求出 $D = \text{rowsum}(dO \circ O)$ (逐元素相乘)

2. 外层循环: 遍历 K 和 V 的块 j 。

● 加载 K 和 V 整个矩阵

3. 内层循环: 遍历 Q 、 O 、 dO 的块 i 。

● 加载 Q_i 、 O_i 、 dO_i dQ 整个矩阵

● 先计算 $S_{ij} = Q_i * K_j^T / \text{sqrt}(d)$ ，得到 S_{ij} 的小块

● 再计算 $P_{ij} = \exp(S_{ij} - L_i)$ ，得到 P_{ij} 的小块

● 之后可以累加得到 $dV_j += P_{ij}^T @ dO_i$ ，即 dV_j 的小块

● 然后求 $dS_{ij} = P_{ij} * (dP_{ij} - D_i)$ ，得到 dS_{ij} 的小块 ($dP_{ij} = dO_i @ V_j^T$)

- dS 有两大用处，一是求 $dK_j+ = dS_{ij}^T @ Q_i$ ，得到 dK_j 的小块，二是求 $dQ_i+ = dS_{ij} @ K_j$ ，得到 dQ_i 的小块

那么，如果把 Q 放在外层会怎么样？

1. 外层循环：遍历 Q 、 O 、 dO 的块 i 。

- 加载 Q_i 、 O_i 、 dO_i 整个矩阵

2. 内层循环：遍历 K 和 V 的块 j 。

- 加载 K 和 V dQ 整个矩阵
- 先计算 $S_{ij} = Q_i * K_j^T / \text{sqrt}(d)$ ，得到 S_{ij} 的小块
- 再计算 $P_{ij} = \exp(S_{ij} - L_i)$ ，得到 P_{ij} 的小块
- 之后可以累加得到 $dV_j+ = P_{ij}^T @ dO_i$ ，即 dV_j 的小块
- 然后求 $dS_{ij} = P_{ij} * (dP_{ij} - D_i)$ ，得到 dS_{ij} 的小块 ($dP_{ij} = dO_i @ V_j^T$)
- dS 有两大用处，一是求 $dK_j+ = dS_{ij}^T @ Q_i$ ，得到 dK_j 的小块，二是求 $dQ_i+ = dS_{ij} @ K_j$ ，得到 dQ_i 的小块

可以看到关键的差别就在 Q_i 、 O_i 、 dO_i 的读取和 K 和 V 的读取上，后者的大小远大于前者，经过这么多次循环后，性能就产生了较大的差距。

对比项	K/V 在外	Q 在外	性能影响
HBM 读取 (K, V)	整个矩阵只被读 1 遍	整个矩阵被读 T_q 遍	灾难性的
HBM 读取 (Q, O, dO)	整个矩阵被读 T_k 遍	整个矩阵只被读 1 遍	占优
核心权衡	T_k 通常远小于 T_q （例如序列长度 8k，块大小 128，则 $T_q=64$ ；如果 K/V 序列长度一样， T_k 也是 64。但在交叉注意力等场景下， T_k 可能很小）。官方方案让被读取次数更多的矩阵 (Q) 在内层循环，被读取次数更少的矩阵 (K/V) 在外层循环。但更关键的是，原子写比重复读更高效。	你的方案让 K 和 V 被反复从 HBM 读取，这是 GPU 计算中最大的性能瓶颈。	
最终评价	最优。最大化了数据在 SRAM 中的复用，将 HBM 的访问降到了最低。	性能极差。导致了对 HBM 的冗余、重复访问，完全违背了 IO 感知的算法设计原则。	

10

分布式并行训练 (Distributed Data Parallel Training)

参考资料 10.1

分布式并行教程 <https://github.com/LambdaLabsML/distributed-training-guide?tab=readme-ov-file>

PyTorch 分布式官方文档 https://docs.pytorch.org/tutorials/beginner/dist_overview.html

cs336 第二篇的主旋律是优化已经实现的训练模型程序的性能；在第一节里我们重点放在了程序本身的优化, 包括如何与硬件协同。

第二节我们优化性能的方式从直观理解上看更加简单粗暴:

我一个人做一件任务要花很长时间, 那么两个人并行分担任务, 那时间自然就降下来了 (但不见得时间就一定变为原来的一半)。这到我们训练的场景上就是先用一块 GPU 训练, 现在用很多块 GPU 一起训练以实现效率提升, 即**分布式并行 (distribute parallel)**。

Extension 10.1 补充知识: 线程 (Thread) 和进程 (Process) 的比较

比较项	进程 (Process)	线程 (Thread)
定义	操作系统下分配资源的基本单位	进程下分配资源的基本单位, 是 CPU 调度 和 执行 的基本单位
比喻	工厂	工人
资源分配	拥有独立的内存地址空间、文件句柄、IO 设备	共享所属进程的绝大部分资源
数据共享	进程之间数据共享困难	线程间通信非常高效
创建开销	创建新进程的资源开销较大	创建新线程的开销较小
稳定性	相互独立, 一个进程崩溃不影响其他进程	一个线程崩溃会影响整个进程及其他线程
依赖关系	进程包含线程, 至少包含一个线程才能执行	线程一定属于某个进程, 无法独立存在

10.1. 单机多进程

顾名思义, 单机多进程是在一个物理服务器 (节点) 上启动多个独立的操作系统进程, 共同协作完成一个计算任务。在现在主流的深度学习训练中, 一般就是每个进程独立的控制一块 GPU (比如 DDP), 但也有一个进程控制多个 GPU 的情况

10.1	单机多进程	121
10.2	AllReduce 算法 . .	123
10.2.1	分布式通信算法 . . .	126
10.3	DDP	130
10.3.1	DDP 和 DP 的区别 . .	130
10.3.2	DDP 算法优化	131
10.4	混 合 并 行 (4D 并 行)	135
10.4.1	每种并行技术简介 . .	135
10.5	混合数据并行 . .	143
10.5.1	DP+PP (数据并行 + 流水 线并行)	143
10.5.2	3D 并行 (DP+PP+TP) .	144
10.5.3	4D 并 行 (DP+PP+TP+ZeRO . .	146
10.5.4	专家并行/MOE 并行 .	148
10.6	CUDA 通信机制知 识补充	148
10.6.1	主 机 与 设 备 (Host-Device) 通信 . .	150
10.6.2	设备内 (Intra-Device) 通信	151

(比如 Model parallelism、Pipeline Parallelism)。

单机多进程是实现数据并行 (DP, Data Parallelism) 的前提, 其核心思想是“**模型复制, 数据分片**”, 每个进程独立完成各自的任务, 再通过**集合通信 (Collective Communication)**来同步梯度, 加速训练。

Extension 10.2 为什么不采用多线程?

规避 Python 的全局解释器锁 (GIL): Python 的 GIL 机制规定, 在同一进程中, 任意时刻只允许一个线程执行 Python 字节码。这使得 Python 的多线程在计算密集型任务中无法实现真正的并行。而多进程模型中, 每个进程都有自己独立的 Python 解释器和内存空间, 因此不受 GIL 的限制, 能够实现真正的并行计算。

下面以讲义上所说的一个简单的单机多进程代码作为示例

这个示例是在 CPU 上进行多进程, 代码的目标是启动 4 个工作进程, 每个进程都生成一个随机整数张量, 然后通过一个名为**all-reduce**的分布式操作, 将这 4 个张量相加, 并将最终的总和结果广播回每个进程。

1. 开局指定 IP 地址和端口号, 指定通信方式是 Gloo(Gloo 既可用于 CPU 也可用于 GPU 通信, 但 GPU 通信性能不如**Nccl**)
2. 每个进程创建一个张量, 进行如下操作:
 - (a). **Reduce (规约)**: 从所有进程收集 data 张量。
 - (b). **Op (操作)**: 对收集到的所有张量执行指定的操作, 这里是 SUM(求和)。
 - (c). **All**: 将计算出的最终结果 (总和) 分发回所有的进程, 并用该结果覆盖它们各自原来的 data。

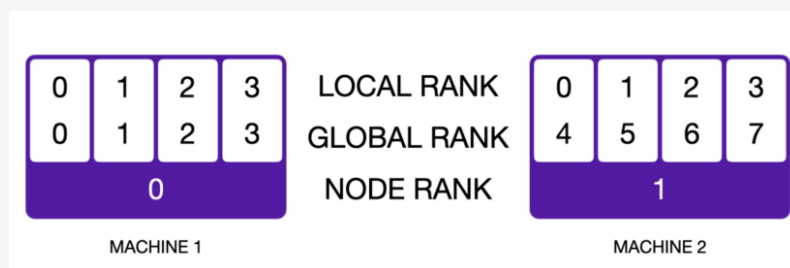


图 10.1: 各种 Rank 之间的关系

Keynote 10.1

区分 *Node Rank*、*Local Rank* 和 *Global Rank*

场景描述: 一个分布式应用运行在 2 台机器 (*MACHINE 1* 和 *MACHINE 2*) 上, *world_size* 为 8(因为 *GLOBAL RANK* 从 0 到 7)。

- **Node Rank (节点排名):** *NODE RANK* 为 0 和 1, 代表这是第 0 号和第 1 号机器。
- **Local Rank (本地排名):** 每台机器上都运行了 4 个进程, 因此在各自的机器内部, 它们的 *LOCAL RANK* 都是从 0 到 3。
- **Global Rank (全局排名):** *GLOBAL RANK* 是全局唯一的。 *MACHINE 1* 上的 4 个进程拥有全局排名 0, 1, 2, 3; 而 *MACHINE 2* 上的 4 个进程则接着拥有全局排名 4, 5, 6, 7。

10.2. AllReduce 算法

参考资料 10.2

AllReduce 算法原理 <https://zhuanlan.zhihu.com/p/79030485>

AllReduce 算法原理 <https://juejin.cn/post/7084135971687497759>

我们理解计算机的算法都是基于一个一个函数操作组合在一起得到的, 那么我们在讲解分布式算法之前, 我们必须先了解一下组成这种算法所应用于硬件的函数操作——集合通信的基本概念,

Broadcast(广播): 将根服务器 (Root Rank) 上的数据分发广播给所有其他服务器 (Rank)

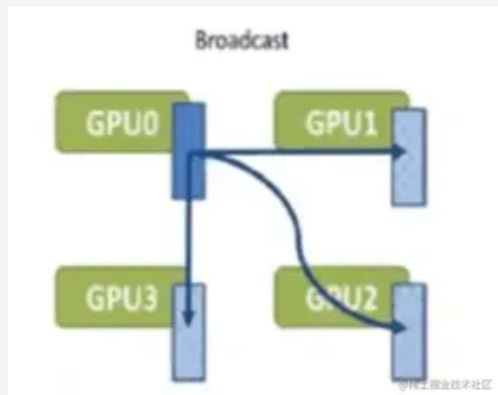


图 10.2: Broadcast

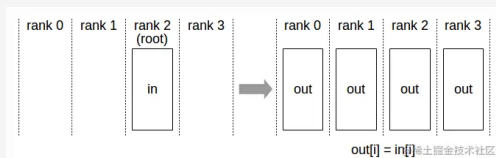


图 10.3: Broadcast

如图所示, 当一台服务器计算完成了自己部分的参数数据, 在分布式训练中想要把自己这部分数据同时发送给其他所有服务器, 那么这种操作方式就叫做广播 (broadcast)。

Scatter(散射): 将根服务器上的数据散射为**同等大小的数据块**, 每一个其他服务器得到一个数据块

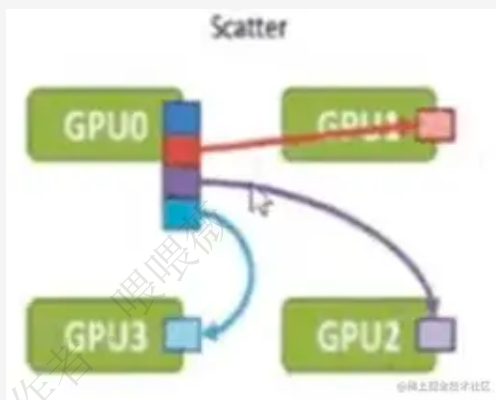


图 10.4: Scatter

如图所示, 当一台服务器计算完成自己部分的参数数据, 但是因为有时候服务器上全部的参数数据过大, 于是我们想要把这台服务器上的数据切分成几个同等大小的数据块 (buffer), 再按照序列 (rank index) 向其他服务器发送其中的一个数据块, 这就叫做散射 (Scatter)。

Gather(聚集): 将其他服务器上的数据块直接**拼接到一起**, 根服务器 (Root Rank) 获取这些数据

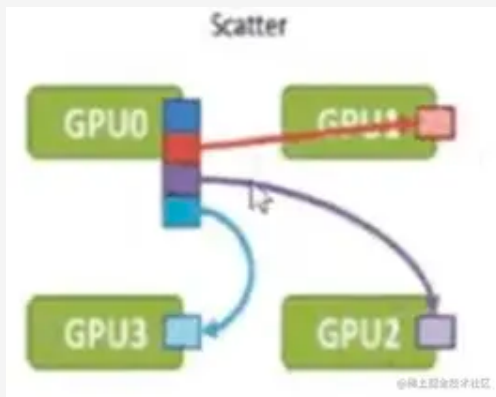


图 10.5: Gather

如图所示, 当服务器都做了散射之后, 每个服务器获得了其他服务器的一个数据块, 我们将一台服务器获得的数据块拼接在一起的操作就叫做聚集 (Gather)。

AllGather(全聚集): 所有的服务器都做上述 Gather 的操作, 于是所有服务器都获得了全部服务器上的数据

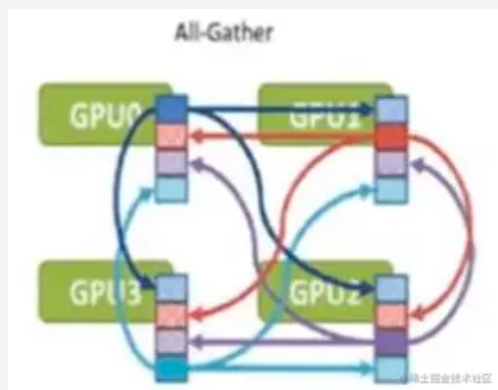


图 10.6: AllGather

如图所示, 所有的服务器都将自己收到的数据块拼接在一起 (都做聚集的操作), 那么就是全聚集 (AllGather)。

Reduce(规约): 对所有服务器上的数据做一个规约操作 (如最大值、求和), 再将数据写入根服务器

如图所示, 当所有服务器都做广播或散射的时候, 我们作为接收方的服务器收到各服务器发来的数据, 我们将这些收到的数据进行某种规约的操作 (常见如求和, 求最大值) 后再存入自己服务器内存中, 那么这就叫规约 (Reduce)。

AllReduce(全规约): 对所有服务器上的数据做一个规约操作 (如最大值、求和), 再将数据写入根服务器

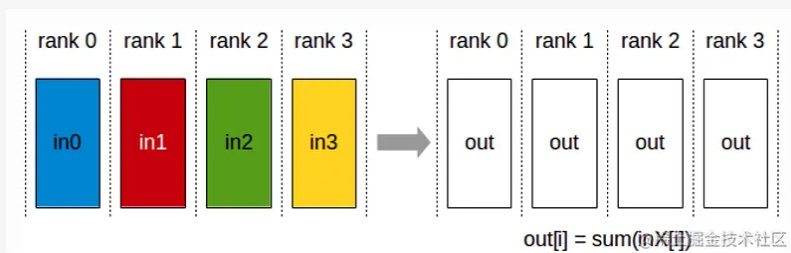


图 10.7: Reduce

如图所示, 同样**每一个服务器都完成上述的规约操作, 那么就是全规约**。这也就是分布式训练最基础的框架, 将所有数据通过规约操作集成到各个服务器中, 各个服务器也就获得了完全一致的、包含原本所有服务器上计算参数的规约数据。

ReduceScatter(散射规约): 服务器将自己的数据分为同等大小的数据块, 每个服务器将根据 index 得到的数据做一个规约操作即, 即先做 Scatter 再做 Reduce。

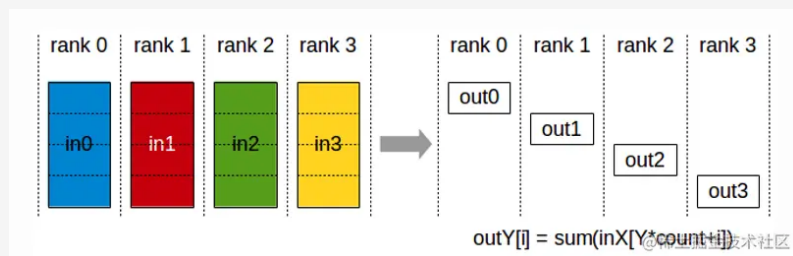


图 10.8: ReduceScatter

简单来讲, 就是先做散射 (Scatter), 将服务器中数据切分成同等大小的数据块, 再按照序列 (Rank Index), 每一个服务器所获得的参数数据做规约 (Reduce)。这就类似于全聚集, 只不过我们将数据不是简单拼接到一起而是做了规约操作 (求和或最大值等操作)。

10.2.1. 分布式通信算法

PS 算法 (Parameter Server, 参数服务器)

PS 算法是一种**中心化**的分布式训练架构。它将计算节点分为两种角色:

- 参数服务器/根服务器 (Parameter Server, PS) 负责**存储**和**更新**模型的全部参数;
- 工作节点 (Worker) 负责使用本地数据**计算梯度**, 并将梯度推送给 PS 做累积 (Reduce), 然后再从 PS 拉取最新的参数。

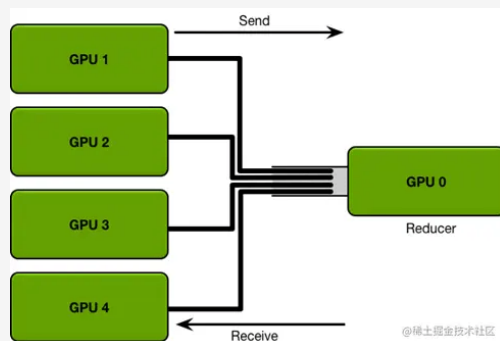


图 10.9: PS 算法示意图

缺点:

- 每一轮的训练迭代都需要所有卡都将数据同步完做一次 Reduce 才算结束, 并行的卡很多的时候,**木桶效应**就会很严重, 一旦有一张卡速度较慢会拖慢整个集群的速度, 计算效率低。
- **Reducer 服务器任务过重**, 成为瓶颈, 所有的节点需要和 Reducer 进行数据、梯度和参数的通信, 当模型较大或者数据较大的时候, 通信开销很大, 根节点收到巨量的数据, 从而形成瓶颈。

Halving and Doubling (HD) 算法 (Recursive Doubling)

阶段一: 规约-散射 (Reduce-Scatter):

1. 在 ' $\log_2(N)$ ' 步中, 节点间的通信距离 (步长) 按 1, 2, 4, ..., $N/2$ 的规律加倍。
2. 第 1 步 (距离=1): 节点 ' i ' 与节点 ' $i+1$ ' 配对。它们交换自己数据数组的一半的**一半**, 并进行累加。例如, 节点 ' i ' 发送前半部分给 ' $i+1$ ', 接收 ' $i+1$ ' 的前半部分并累加; 同时发送后半部分, 接收后半部分并累加。完成后, 节点 ' i ' 保留累加的前半部分, 节点 ' $i+1$ ' 保留累加的后半部分。
3. 第 2 步 (距离=2): 节点 ' i ' 与节点 ' $i+2$ ' 配对。它们交换的数据块是上一步规约后的结果, 数据量仍然是数组的一半。
4. ... 依此类推...
5. 阶段结果: 经过 ' $\log_2(N)$ ' 步后, 每个节点都只拥有整个数组的 ' $1/N$ ', 但这 ' $1/N$ ' 的部分是所有节点对应部分的全局规约结果。

阶段二: 全收集 (All-Gather):

1. 这个阶段是第一阶段的**逆过程**。
2. 在 ' $\log_2(N)$ ' 步中, 节点间的通信距离按 $N/2$, ..., 4, 2, 1 的规律减半。
3. 第 1 步 (距离= $N/2$): 在第一阶段最后一步通信的节点对 (例如 ' i ' 和 ' $i+N/2$ ') 互相交换它们持有的 ' $1/N$ ' 的最终结果。现在它们各自拥有了 ' $2/N$ ' 的全局结果。
4. ... 依此类推...
5. 最终结果: 经过 ' $\log_2(N)$ ' 步的逆向操作, 每个节点都逐步从邻居那里收集到了所有的最终数据块, 最终恢复出完整的全局规约结果。

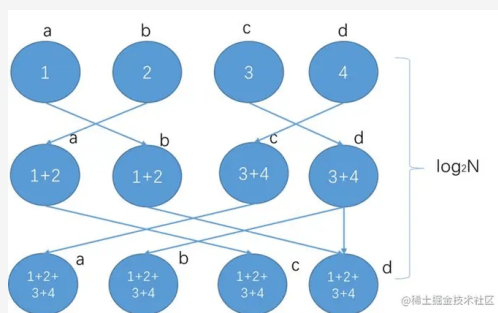


图 10.10: HD 算法示意图

Ring 算法 (Ring-AllReduce)

核心思想: 将数据分成 N 个块 (chunk), 通过两个阶段, 每个阶段 ' $N-1$ ' 步, 完成规约和分发。

工作流程:

阶段一: 散射-规约 (Scatter-Reduce):

1. 分块: 每个节点都将自己的数据数组分成 N 个小块。
2. 传递与累加: 在 ‘ $N-1$ ’ 步中, 进行循环传递。
3. 在第 ‘ k ’ 步, 每个节点 ‘ i ’ 会将自己的第 ‘ $(i-k+N)$ ’
4. 同时, 它会从上一个节点 ‘ $(i-1+N)$ ’
5. 阶段结果: 经过 ‘ $N-1$ ’ 步后, 每个数据块都完整地绕环一周, 并累加了所有节点上对应位置的值。此时, 每个节点 ‘ i ’ 都拥有一个最终的、全局规约后的数据块 (具体来说是第 ‘ $(i-1+N)$ ’)

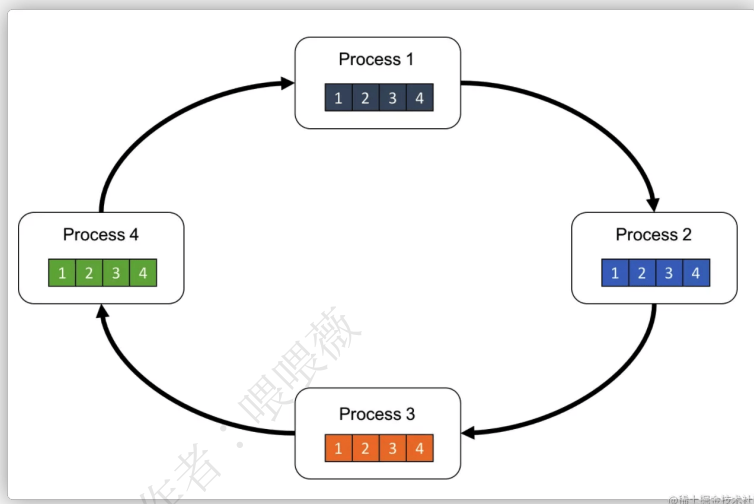


图 10.11: Ring 算法示意图 1

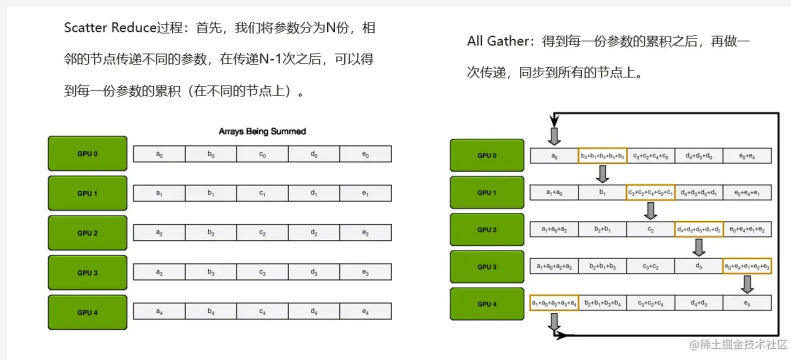


图 10.12: Ring 算法示意图 2

阶段二: 全收集 (All-Gather):

1. 再次传递: 现在每个节点都持有一个“最终版”的数据块, 目标是让所有节点都拥有所有 N 个“最终版”数据块。
2. 简单传递: 在接下来的 ‘ $N-1$ ’ 步中, 节点们只是单纯地传递它们手中的“最终版”数据块, 不再进行累加。

3. 最终结果: 经过 ' $N-1$ ' 步后, 所有最终块都在环上完整地跑了一圈, 每个节点都收集到了所有 N 个最终块, 从而得到了完整的全局规约结果。

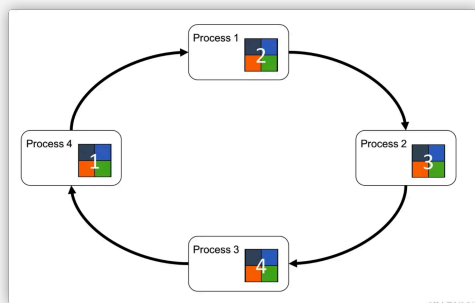


图 10.13: Ring 算法示意图 3

性能特点:

1. 延迟: 总共需要 ' $2 * (N-1)$ ' 次通信步骤, 延迟与节点数 ' N ' 成正比。
2. 带宽: 在任何时刻, 每个节点只发送和接收一小块数据 (大小为 ' M/N '). 算法可以使链路带宽持续被占满, 因此带宽利用效率非常高。它被认为是**带宽最优 (Bandwidth Optimal)**的。现代的 pytorch 的 DDP 并行的底层计算也是靠 Ring 算法。

10.3. DDP

参考资料 10.3

DDP 原理 https://blog.csdn.net/my_name_is_learn/article/details/146468992

桶排序算法 <https://www.runoob.com/w3cnote/bucket-sort.html>

10.3.1. DDP 和 DP 的区别

DDP(Distribute Data Parallel) 是在 DP(Data Paralle) 的基础上的进一步优化。两者的主要特性的对比:

特性	DP (DataParallel)	DDP (DistributedDataParallel)
底层实现	单进程多线程	多进程 (通常一个 GPU 一个进程)
模型复制	每个前向传播, 主 GPU 上的模型会被复制到所有辅助 GPU。	每个进程维护一个独立的模型副本。
数据处理	主 GPU 将输入数据分割, 并分发给所有 GPU。	每个进程独立地加载数据, 或从数据加载器获取其应处理的数据分片。
前向、反向传播	每次传播前, 主 GPU 将模型分给其他 GPU, 计算完了之后其他 GPU 再把结果返回给主 GPU, 不保存模型。	每个 GPU 都有保有独立的模型, 独立进行计算。
计算负载	不均衡 。主 GPU 承担额外的任务 (如收集输出、计算 Loss、计算梯度、更新参数)。	均衡 。每个进程独立完成前向、反向传播和参数更新。
梯度同步	反向传播在主 GPU 上完成, 主 GPU 更新模型。	每个进程计算本地梯度, 通过 All-Reduce 在所有进程间同步并平均梯度。
通信效率	效率较低, 模型和数据在主 GPU 和辅助 GPU 之间频繁传输。每个前向传播都需要复制模型。	效率较高, 只在反向传播时同步梯度 (相对较小的数据量)。利用高效的通信库。
GIL 影响	受 Python 全局解释器锁 (GIL) 的潜在影响。	由于是多进程, 不易受 GIL 影响。
适用场景	单机多卡, 代码简单, 快速验证小模型。	单机多卡、多机多卡, 大规模模型训练, 追求高性能和更好的可伸缩性。
pytorch 代码	<code>torch.nn.DataParallel</code>	<code>torch.nn.parallel.DistributedDataParallel</code>

DDP 的工作流程可以概括为以下几个步骤:

1. 初始化

- **多进程启动**:DDP 采用多进程模型, 为每个 GPU 分配一个独立的进程。这与 Python 的**全局解释器锁 (GIL)**完美契合, 因为不同进程拥有独立的内存空间和 Python 解释器, 可以实现真正的并行计算。
- **进程组建立**: 通过 ‘`torch.distributed.ini_process_group`’函数初始化一个进程组 (Process Group), 组内的所有进程共同参与训练。每个进程被分配一个唯一的排名 (rank), 从 0 到 N-1(N 是总 GPU 数)。通常,rank 0 被指定为主进程 (master)。

- **模型复制**: 使用广播集合通信操作 (broadcast) 将模型参数从 rank 0 的设备发送到所有其他设备。在训练开始时, 每个设备都持有相同的模型参数和优化器状态副本 (例如 Adam 优化器中累积的梯度统计量)。

2. 数据分发

- **数据分发**: 给定一个包含 n 个样本的批次时, 该批次会被分片处理, 每个设备接收 n/d 个互不重叠的样本 (其中 d 是用于数据并行训练的设备数量)。 n 必须能被 d 整除, 因为训练时间受限于最慢进程的速度 (如果不能整除, 那么就会有某一/几个设备的数量大于其他设备, 导致不同设备之间时间不一致, 产生**木桶效应**)

3. 计算与梯度同步 (DDP 的精髓)

- **前向传播**: 每个进程独立地在其分配到的数据批次上执行模型的前向传播, 计算损失。
- **反向传播与梯度同步**: 当调用 `loss.backward()` 时, 梯度从模型的输出层向输入层逐层计算 (这里会有**梯度分桶**、**计算通信重叠**等优化操作, 后面会细说), 每个反向传播进程计算得到的梯度都会保存。
- **All-Reduce 操作**: 这是去中心化的梯度同步算法。在 All-Reduce 操作中, 每个进程都会贡献出自己计算出的梯度, 并最终从操作中获得所有进程梯度的总和 (或平均值)。最常用的高效实现是**Ring-AllReduce**。

4. 模型更新

- **同步更新**: All-Reduce 操作完成后, 每个进程都拥有了完全相同的平均梯度。
- **本地更新**: 每个进程独立调用优化器 (如 `optimizer.step()`) 来更新其本地模型副本的权重。因为初始权重相同, 更新的梯度也相同, 所以更新后的模型权重在所有进程间依然保持严格一致。

10.3.2. DDP 算法优化

主要涉及了两种重要的方法,**梯度分桶**和**计算通信重叠**。

之前朴素实现 DDP 的性能瓶颈:

1. **通信开销大**: 它为模型中的每一个参数张量都单独执行一次 `all-reduce` (全局规约) 通信操作。频繁的通信调用会带来显著的性能开销。
2. **通信与计算未重叠**: 它需要等待整个反向传播过程计算完所有梯度后, 才开始进行通信, 这浪费了宝贵的计算资源和时间。

针对第一个瓶颈, 容易想到为了避免用循环每次都要全规约进行通信, 我们可以将循环里的所有的梯度张量先存起来, 等循环结束后再一起发送。这样做的好处是减少了通信的开销, 但缺点是没能充分发挥**“并行”**和**“异步”**的力量。

具体而言, 由于反向传播不止计算一个参数的梯度, 我们完全可以等一个参数梯度计算好了之后, 让它**立刻异步执行 all_reduce**, 然后再去算下一个参数的梯度, 这样类似流水线的做法充分发挥了**“并行”**的能力, 理论上更省时间。

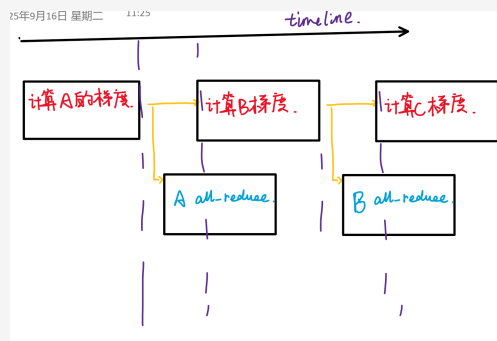


图 10.14: 计算通信重叠

在具体实现里面, 反向传播的代码 `loss.backward()` 比较固定, 但 `pytorch` 也配备了 `hook`(钩子) 操作。钩子是一种允许你“挂载”自定义代码到现有程序执行流程中的一种机制。它使得你可以在不修改程序或框架源代码的情况下, 对程序的行为进行监视、修改或增强。它本质上是一种回调 (`callback`) 思想的体现: 你预先注册一个函数, 然后由系统在某个特定事件发生时“回头调用”它。

因此, 我们可以在反向传播完一个参数之后利用 `hook` 操作异步进行 `all_reduce`, 从而实现计算和通信的流水线重叠。

```
for param in reversed(list(self.module.parameters())): # 这里由于是反向传播, 梯度从后往前计
    ↪ 算, 所以加一个 reverse
    if param.requires_grad:
        param.register_post_accumulate_grad_hook(self._create_hook(param))
```

接下来介绍分桶排序优化的方法, 它更好结合了两种方法缓解两个瓶颈。

相比于之前的每一个参数一计算完就进行通信的方式, 利用桶排序对计算好的参数分批进行通信的方式能显著减少通信次数, 从而减少网络压力, 而且利用桶的方式有更好地灵活性。比方说某个任务一定要追求极低的时延, 那就可以把桶设置成一个桶只装一个参数的形式, 这就变成了之前的这种形式, 如果某个任务所处的网络环境不好, 不方便经常通信, 那就把桶设置的大一些, 这样就减少了通信次数。

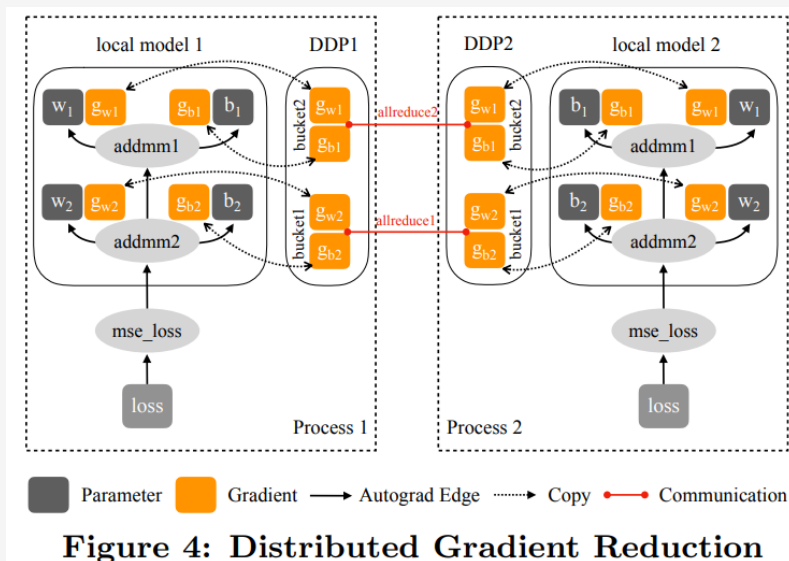


图 10.15: 桶排序流程

下面介绍整体的流程:

1. 初始化阶段: 桶的构建 (Bucket Construction)

- **参数排序**: DDP 首先会拿到模型的所有可训练参数 (`model.parameters()`)。它会按照这些参数在反向传播中梯度被计算的**逆序**来排列它们。(逆序至关重要, 因为反向传播是从模型的最后一层开始, 逐层向前计算梯度的。因此, 排在最前面的参数 (例如模型最后一层全连接层的权重和偏置) 是反向传播时最先准备好梯度的。)
- **参数入桶**: DDP 会遍历这个逆序的参数列表, 依次将参数装入桶中。每个桶有一个预设的**容量上限**, 由参数 `'bucket_cap_mb'` 控制 (默认为 25MB)。DDP 会持续向当前桶中添加参数, 直到加入下一个参数会导致桶的总大小超过容量上限为止。此时, 当前桶构建完毕, DDP 会新建一个空桶, 并将刚才那个放不下的参数作为新桶的第一个成员, 然后继续该过程, 直到所有参数都被分配到桶中。
- **结果**: 初始化完成后, DDP 内部就维护了一个桶的列表, 每个桶里包含了一组参数。例如, 对于一个典型的 Transformer 模型, 最后一个 `'Linear'` 层和 `'LayerNorm'` 层的参数可能在同一个桶里 (桶 0), 倒数第二个注意力块的参数可能在桶 1, 以此类推。

2. 反向传播阶段: Autograd Hook 的触发

- **注册钩子 (Registering Hooks)**: 在初始化阶段, DDP 会遍历模型的所有参数, 并为每一个参数的梯度 (`'grad'` 属性) 注册一个回调函数, 即 `'autograd'` 钩子。
- **触发机制**: 当你调用 `'loss.backward()'` 时, PyTorch 的 `'autograd'` 引擎会负责计算每个参数的梯度。每当一个参数的梯度被完全计算出来后, `'autograd'` 引

擎会**立即自动调用**之前注册在它上面的那个钩子函数。

3. 核心逻辑: 标记、检查与异步通信

- **标记就绪**: 钩子函数首先会找到该梯度所属的桶, 并在桶内部的计数器或状态中, 将这个梯度标记为“已就绪”(Ready)。
- **检查桶满**: 接下来, 钩子会检查这个桶内是否所有参数的梯度都已处于“已就绪”状态。
- **触发通信**:
 - 如果桶内还有其他参数的梯度没算好, 钩子函数不做任何事, 直接返回。继续进行下一轮的梯度计算。
 - 如果桶内所有参数的梯度都算好了, DDP 会执行以下操作:
 - **梯度合并 (Flatten)**: 为了提高通信效率, DDP 会将这个桶里所有零散的梯度张量复制并拼接成一个巨大、扁平且在内存上连续的单一张量。这被称为‘flatten’操作, 因为单次传输一个大 Tensor 远比多次传输多个小 Tensor 要快。
 - **启动异步 All-Reduce**: DDP 会立即对这个合并后的大 Tensor 调用‘torch.distributed.all_reduce(..., async_op=True)’。告诉 NCCL 开启异步通信, 这边不阻塞继续下一个桶的梯度计算。

4. 等待与梯度回写

- **等待与同步**: 在整个‘loss.backward()’的末尾, DDP 会确保所有桶的 All-Reduce 操作都已完成。它通常只需要等待最后一个被触发的桶的通信完成即可, 因为前面的通信已经和计算重叠了。
- **梯度解包与回写 (Unflatten)**: 当一个桶的 All-Reduce 操作完成后, 其对应的扁平化 Tensor 中存储的是该桶所有梯度拼接后的结果。DDP 会将其除以分布式环境的‘world_size’(即 GPU 总数)来得到平均梯度。然后, 再将这个扁平 Tensor 中的值“解包”, 按顺序写回到该桶内每个原始参数的‘grad’属性中。
- **优化器更新**: 至此, 所有参数的‘grad’属性都包含了同步后的全局平均梯度, 优化器(‘optimizer.step()’)便可以安全地使用它们来更新模型权重了。

10.4. 混合并行 (4D 并行)

参考资料 10.4

DeepSeed 原文 <https://www.microsoft.com/en-us/research/blog/deepspeed-extreme-scale-model-training-for-everyone/>

DeepSeed 文档 <https://www.deepspeed.ai/tutorials/pipeline/>

混合并行教程 https://blog.csdn.net/qq_25295605/article/details/143671968

混合并行教程 <https://huggingface.co/docs/transformers/v4.15.0/en/parallelism>

张量模型并行 <https://zhuanlan.zhihu.com/p/622212228>

混合并行技术是指同时使用多种并行技术，比如数据并行 (DP) 和张量并行 (TP)，或者数据并行和流水线并行 (PP)。

10.4.1. 每种并行技术简介

数据并行 (Data Parallelism, DP)

最常见的一种并行方式。它将同一个模型完整地复制到多块 GPU 上。然后，将一大批训练数据切分成多份小数据，每块 GPU 拿到一份小数据独立进行计算。在每个训练步骤结束时，所有 GPU 会同步一次计算结果 (通常是梯度)，以确保所有模型副本的参数保持一致。

DP 工作流程:

1. 数据切分:

- 在一个训练循环中，你从数据加载器中取出一个大的 ‘batch’。
- 主 GPU (通常是 ‘cuda:0’) 会将这个大 ‘batch’ 沿着批次维度进行切分，分成 N 个子批次，其中 N 是参与训练的 GPU 数量。
- 然后，主 GPU 将这些子批次分发到其他各个 GPU 上 (包括它自己)。

2. 模型复制:

- 在每次前向传播开始之前，主 GPU (‘cuda:0’) 上的最新模型参数会被复制并广播 (broadcast) 到所有其他的从属 GPU 上，确保每个 GPU 都使用完全相同的模型。

3. 并行计算:

- 每个 GPU 拿到自己的子批次数据和复制过来的模型。
- 所有 GPU **并行地、独立地** 执行前向传播，计算出各自的损失。
- 接着，每个 GPU 根据自己的损失，并行地、独立地执行后向传播，计算出模型参数的梯度。

4. 梯度规约 (Gather & Reduce):

- 所有从属 GPU 将它们计算出的梯度全部发送回**主 GPU**（‘cuda:0’）。
- 主 GPU 负责收集所有 GPU 传来的梯度，并将它们进行**Reduce(取平均)**操作。

5. 模型参数更新:

- 主 GPU 使用规约后的总梯度，通过优化器来更新其自身的模型参数。
- 请注意：**只有主 GPU 上的模型参数得到了更新**。其他 GPU 上的模型在此刻还是旧的。

6. 模型广播 (Broadcast):

- 为了开始下一次迭代，主 GPU 会将刚刚更新完毕的、最新的模型参数再次广播给所有从属 GPU。
- 循环回到第 1 步，处理下一个 ‘batch’ 的数据。

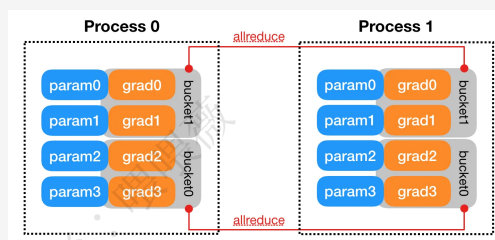


图 10.16: DP 并行示意图

张量并行 (TensorParallel, TP):

这种方式用于解决**模型本身过大、单块 GPU 无法容纳**的问题。它不对模型进行复制，而是将模型中的某个大张量（例如一个巨大的权重矩阵）切分成多个小块 (Shard)，每一块分别存放在不同的 GPU 上。计算时，每块 GPU 只处理自己负责的那一小块张量，最后再将结果同步汇总。

举个例子，对于一个神经网络计算单元：‘ $Y = X * W$ ’

- ‘X’ 是输入激活 (Input Activation)
- ‘W’ 是权重矩阵 (Weight Matrix)
- ‘Y’ 是输出激活 (Output Activation)

如果权重矩阵 ‘W’ 太大，无法放入单个 GPU，我们就需要将 ‘W’ 切分到多个 GPU 上，并设计一种方法让这些 GPU 协同完成 ‘ $X * W$ ’ 这个运算。

张量并行的具体实现方式:

对于张量的切分也是有讲究的，如果随便切分很难实现 ‘ $X * W$ ’ 的同步运算，因此 ‘W’ 的切分往往是按照行或者按照列

列并行 (Column Parallelism):

我们将权重矩阵 ‘W’ **按列**切分。假设我们有 2 个 GPU:

$$W = [W_1 \mid W_2]$$

其中 ‘ W_1 ’ 和 ‘ W_2 ’ 分别是 ‘ W ’ 的左半部分和右半部分, 它们被分发到 GPU 1 和 GPU 2。

1. 前向传播 (Forward Pass):

输入 ‘ X ’ 被复制到两个 GPU 上。GPU 1 计算 ‘ $Y_1 = X * W_1$ ’, GPU 2 计算 ‘ $Y_2 = X * W_2$ ’。计算完成后, 我们将结果拼接起来: ‘ $Y = [Y_1 | Y_2]$ ’。注意, 在这个过程中, ‘ Y ’ 的不同部分自然地分布在不同的 GPU 上。

2. 通信: 在这一步的前向传播中, **不需要**任何通信。每个 GPU 独立计算自己那一部分。

3. 反向传播 (Backward Pass):

- 在计算输入的梯度 ‘ dL/dX ’ 时, 根据链式法则, 它等于 ‘ $(dL/dY) * W^T$ ’。
- 由于 ‘ Y ’ 和 ‘ W ’ 都是分片的, ‘ dL/dX ’ 的计算会变成 ‘ $(dL/dY_1) * W_1^T + (dL/dY_2) * W_2^T$ ’。
- GPU 1 计算 ‘ $(dL/dY_1) * W_1^T$ ’, GPU 2 计算 ‘ $(dL/dY_2) * W_2^T$ ’。
- 为了得到最终完整的 ‘ dL/dX ’, 两个 GPU 需要将各自计算出的梯度相加。

4. 通信: 这里需要一次 **All-Reduce** 操作, 将所有 GPU 上的梯度累加, 并同步回每个 GPU。

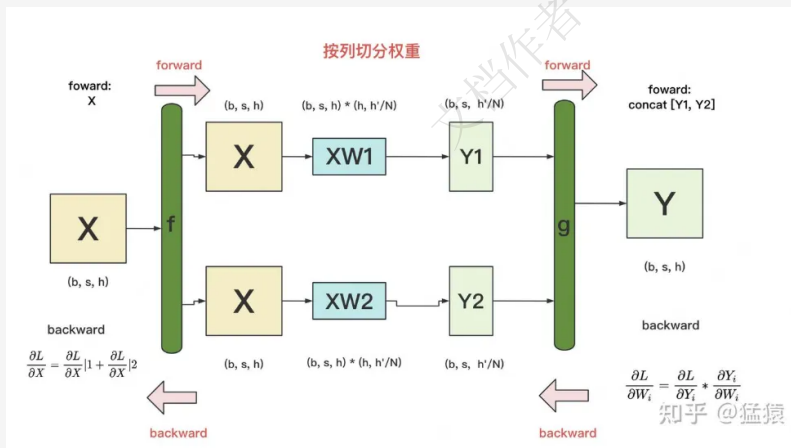


图 10.17: 列并行示意图

行并行 (Row Parallelism):

现在, 我们将权重矩阵 ‘ W ’ **按行**切分:

‘ $W = [W_1; W_2]$ ’ (这里用分号表示按行堆叠)

‘ W_1 ’ 和 ‘ W_2 ’ 分别是 ‘ W ’ 的上半部分和下半部分, 被分发到 GPU 1 和 GPU 2。

1. 前向传播 (Forward Pass):

此时, 输入 ‘ X ’ 必须是已经被分片的 (例如, 它是上一个列并行层的输出)。假设 ‘ $X = [X_1 | X_2]$ ’。

完整的计算是 $Y = X * W = [X_1 | X_2] * [W_1; W_2] = X_1 * W_1 + X_2 * W_2$ 。GPU 1 计算 $Y_1 = X_1 * W_1$ 。GPU 2 计算 $Y_2 = X_2 * W_2$ 。为了得到最终的 Y ，我们需要将 Y_1 和 Y_2 相加。

2. **通信**: 这里需要一次 **All-Reduce 操作**, 将各个 GPU 上的部分结果相加, 得到最终的完整输出 Y 。
3. **反向传播 (Backward Pass)**: 在计算输入的梯度 dL/dX 时, 它的一部分 dL/dX_1 等于 $(dL/dY) * W_1^T$, 这可以在 GPU 1 上独立完成。
4. **通信**: 在这一步的反向传播中, **不需要** All-Reduce 通信。

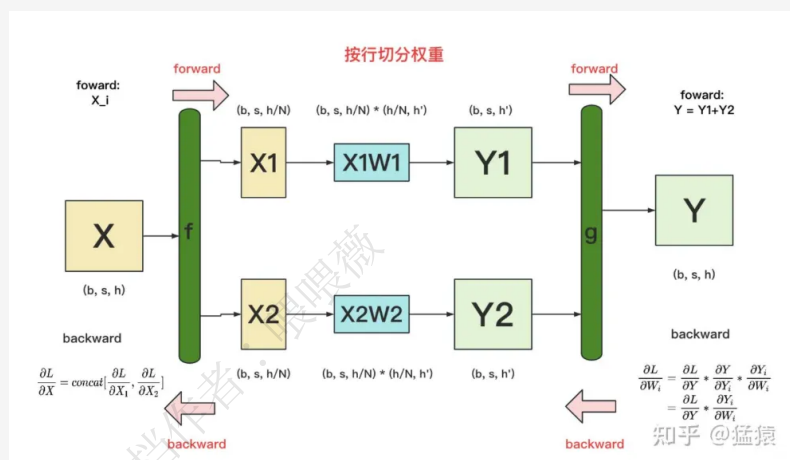


图 10.18: 行并行示意图

行列结合:

以 FFN 为例, 它通常由两个线性层和一个激活函数组成: $Y = \text{GELU}(X * A) * B$

1. **第一个线性层 ($X * A$):** 使用列并行。

- A 按列切分 $[A_1, A_2]$ 。
- 输出 $\text{GELU}(X * A)$ 也是按列切分的 $[Y_1, Y_2]$, 分布在不同 GPU 上。
- 前向传播**无通信**。反向传播需要一次 **All-Reduce**。

2. **第二个线性层 ($\dots * B$):** 使用行并行。

- B 按行切分 $[B_1; B_2]$ 。
- 它的输入 $[Y_1, Y_2]$ 恰好是上一步的分布式输出。
- 前向传播需要一次 **All-Reduce** 来合并最终结果。反向传播**无通信**

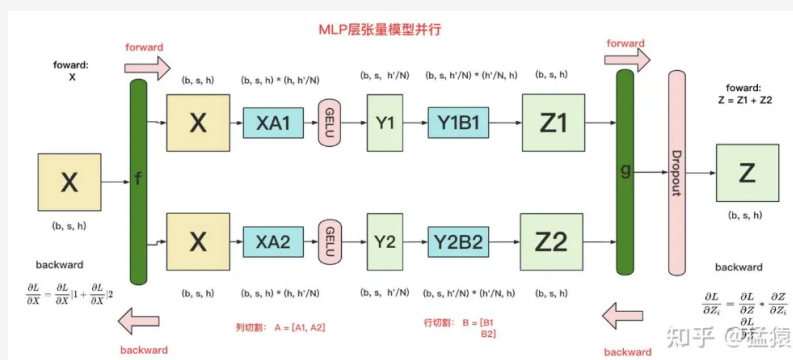


图 10.19: 行列结合示意图

流水线并行 (PipelineParallel, PP)

这种方式也是为了解决模型过大的问题, 但它的切分维度不同。它将整个模型**按层 (Layer) 进行切分**, 类似于工厂里的流水线。例如, 一个 30 层的模型, 可以把前 10 层放在 GPU 1 上, 中间 10 层放在 GPU 2 上, 最后 10 层放在 GPU 3 上。数据像在水流线上传送带一样, 依次流过这几块 GPU, 完成一次完整的计算。

基本原理:

假设我们将一个模型切分成 4 个部分, 分别放在 GPU 0, 1, 2, 3 上。

前向传播 (Forward Pass):

1. ‘Batch 1’ 进入 ‘GPU 0’ 进行计算。
2. ‘GPU 0’ 计算完成后, 将其输出 (称为激活值) 发送给 ‘GPU 1’。
3. ‘GPU 1’ 收到后开始计算, 计算完成后将激活值发送给 ‘GPU 2’。
4. ... 以此类推, 直到 ‘GPU 3’ 完成计算。

后向传播 (Backward Pass):

1. ‘GPU 3’ 首先计算梯度。
2. ‘GPU 3’ 将其计算的梯度发送回 ‘GPU 2’。
3. ‘GPU 2’ 接收到梯度后, 计算自己的梯度, 再发送回 ‘GPU 1’。
4. ... 以此类推, 直到 ‘GPU 0’ 完成梯度计算。

Keynote 10.2

流水线并行的缺陷——巨大的“气泡” (Bubble)

在上述流程中, 当 ‘GPU 1’ 在工作时, ‘GPU 0’ 在做什么? 它在空闲等待。当 ‘GPU 2’ 工作时, ‘GPU 0’ 和 ‘GPU 1’ 都在空闲等待。在任何一个时间点, 只有一个 GPU 在忙碌, 其他的 GPU 都在闲置。这种 GPU 利用率的巨大浪费, 我们称之为流水线气泡 (Pipeline Bubble)。

优化: 微批次流水线

为了解决“气泡”问题，现代流水线并行引入了一项关键技术：**将一个大的训练批次 (mini-batch) 进一步切分成若干个更小的微批次 (micro-batch)**。

通过这种方式，虽然在开始和结束阶段仍有无法避免的“气泡”，但中间大部分时间 GPU 的利用率被极大地提高了。微批次数量越多，稳态阶段占比就越大，“气泡”的相对开销就越小。

梯度累积：在后向传播阶段，每个 GPU 会计算出自己所负责模型部分的梯度。它会累积所有微批次的梯度，直到整个 mini-batch 处理完毕，然后用累积的总梯度进行一次参数更新

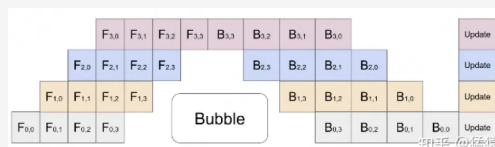


图 10.20: 流水线并行示意图

ZeRO (Zero Redundancy Optimizer)

这是一种**旨在消除数据并行中的冗余、极致节省显存**的技术。它类似于张量并行，也会对模型的参数、梯度和优化器状态进行分片。但它的核心特点是：只有在计算需要时，才会动态地将完整的张量临时重建出来，计算一完成就立即丢弃，从而大大降低了显存峰值。它还支持将暂时用不到的数据“卸载 (Offload)”到 CPU 内存或硬盘上，进一步节省宝贵的 GPU 显存。

DDP 的内存冗余问题

在标准的数据并行训练中，每个 GPU 都需要存储三份主要的模型相关数据：

1. **模型参数 (Model Parameters)**：模型的权重，例如 ‘Float16’ 或 ‘Float32’。
2. **梯度 (Gradients)**：反向传播计算出的参数梯度，大小与参数相同。
3. **优化器状态 (Optimizer States)**：例如 Adam 优化器需要为每个参数存储其一阶矩 (**Momentum**) 和二阶矩 (**Variance**)，通常是参数数量的 2 倍。

假设模型参数数量为 M，使用 Adam 优化器和 FP16 混合精度训练，那么在 ‘N’ 个 GPU 上进行数据并行时，每个 GPU 的内存占用大约是：

‘Memory_per_GPU ≈ (2M for FP16 Params) + (2M for FP16 Grads) + (4M for FP32 Momentum) + (4M for FP32 Variance) = 12M’ (这还不包括激活值等其他开销)

总的内存占用是 ‘N * 12M’。你会发现，模型参数、梯度和优化器状态在所有 ‘N’ 个 GPU 上都存在一份完整的副本，这是巨大的**内存冗余**。ZeRO 的使命就是消除这种冗余。

ZeRO 的核心思想：分区 (Partitioning)

ZeRO 通过将这三部分数据 (P, G, O) 在所有参与数据并行的 GPU 之间进行**分区**，而不是复制，来解决内存冗余问题。每个 GPU 只负责存储和更新完整数据的一个分片 (Shard)。

ZeRO 分三个递进的阶段来实现这一目标, 每个阶段优化的程度都比前一个更深。

Stage 1: 优化器状态分区 (Partition Optimizer States)

优化器状态通常是内存的大头 (在 Adam 中是参数量的 2 倍, 且通常用 FP32 存储, 总计 8M)。这是最容易优化的部分。

工作流程:

1. **分区**: 将优化器状态 (如 Momentum 和 Variance) 平均切分成 ‘N’ 份, 每个 GPU 只保存其中的 ‘1/N’。
2. **保留**: 每个 GPU 依然保留完整的模型参数和梯度。
3. **更新过程** (`optimizer.step()`):
 - 在反向传播结束后, 所有 GPU 上的梯度会通过一次 ‘All-Reduce’ 操作进行同步, 确保每个 GPU 都有完整的、求和后的梯度。
 - 在参数更新时, 每个 GPU 只使用它本地存储的那一小部分优化器状态, 来更新对应的那一小部分模型参数。
 - 更新完成后, 每个 GPU 只拥有了模型参数的一部分更新。
 - 最后, 通过一次 ‘All-gather’ 通信操作, 所有 GPU 广播自己更新好的那部分参数, 从而在每个 GPU 上重新组合出完整的、更新后的模型。

效果:

- **内存节省**: 大幅减少了优化器状态的内存占用。
- **通信开销**: 与标准 DDP 相比, 增加了一次 ‘All-gather’ 操作来同步更新后的参数。

Stage 2: 梯度和优化器状态分区 (Partition Gradients & Optimizer States)

在 Stage 1 的基础上, 进一步消除梯度的冗余。

工作流程:

1. **分区**: 将优化器状态**和梯度**都进行 ‘N’ 路分区。每个 GPU 只负责 ‘1/N’ 的梯度和优化器状态。
2. **保留**: 每个 GPU 依然保留完整的模型参数。
3. **更新过程**:
 - 在反向传播过程中, 当梯度计算出来后, 不再执行 ‘All-Reduce’。取而代之的是一个更高效的 ‘Reduce-Scatter’ 操作。这个操作会一边计算梯度总和, 一边将结果直接分发给对应的 GPU。例如, 属于分区 1 的梯度会被计算总和后直接发送到 GPU 1, 以此类推。
 - 这样, 反向传播结束后, 每个 GPU 上只存储了它所负责的那部分梯度总和。
 - 后续的优化器更新和参数 ‘All-gather’ 过程与 Stage 1 类似。

效果:

- **内存节省**: 进一步节省了梯度存储。
- **通信效率**: 将 DDP 中的 ‘All-Reduce’ 替换为 ‘Reduce-Scatter’, 通常通信量减半, 效率更高。

Stage 3: 参数、梯度和优化器状态全部分区 (Partition Everything)

这是最极致的优化, 消除了所有冗余, 包括模型参数本身。

工作流程:

1. **分区**: 将模型参数、梯度、优化器状态**全部**进行 ‘N’ 路分区。
2. **保留**: 原则上, 每个 GPU 在任何时刻都只拥有模型的一个分片。
3. **计算过程 (前向/后向)**:
 - 当一个 GPU 需要执行某一层的前向或后向计算时, 如果该层所需的参数不在本地, 它需要通过一次 ‘All-gather’ 操作从其他 GPU 那里动态地获取这些参数。
 - 计算完成后, 为了节省内存, 这些临时的、非本地的参数可以被立即丢弃。
 - 这个过程需要在每一层计算前后动态地进行, 对通信的要求非常高。

效果:

- **内存节省**: 达到理论最优。理论上可以将一个巨大的模型 (如 1 万亿参数) 分散到足够多的 GPU 上进行训练。
- **通信开销**: 通信量巨大, 因为它在计算过程中穿插了大量细粒度的 ‘All-gather’ 操作。

ZeRO 的扩展: ZeRO-Offload & ZeRO-Infinity

为了应对 GPU 内存仍然不足或通信成本过高的情况, ZeRO 还发展出了更高级的形态:

- **ZeRO-Offload**: 将 ZeRO 分区后的数据 (特别是那些不常被访问的, 如优化器状态) 进一步从 GPU 内存**卸载 (Offload)**到 CPU 内存。这极大地扩展了可训练模型的大小, 代价是引入了 GPU 和 CPU 之间的 PCIe 通信延迟, 速度会慢一些。
- **ZeRO-Infinity**: ZeRO-Offload 的终极版本, 不仅利用 CPU 内存, 还利用 NVMe 固态硬盘作为海量的内存池。这使得在有限的硬件上训练万亿级参数模型成为可能, 尽管训练时间会非常长。

10.5. 混合数据并行

10.5.1. DP+PP(数据并行 + 流水线并行)

为什么需要 DP+PP ?

- **单独使用数据并行 (DP)**: 要求**每个 GPU**都能装下完整的模型。当模型增长到百亿甚至千亿参数时, 例如一个 175B 的 GPT-3 模型 (FP16 下需要 350GB 内存), 远超任何单个 GPU 的容量, DP 就失效了。
- **单独使用流水线并行 (PP)**: 解决了模型过大的问题, 可以将模型切分到多个 GPU 上。但它的扩展性受限于模型的层数和“**流水线气泡**”的开销。如果你只有 60 层模型, 使用超过 60 个 GPU 做流水线并行是没有意义的, 而且气泡开销会变得极其巨大。它无法利用更多的机器来处理更多的数据。

DP+PP 的核心思想: 用 PP 来解决单个模型过大的问题, 用 DP 来将这个“切分后的大模型”复制多份, 以扩展到更多的计算设备上, 实现规模化加速。

DP+PP 的工作流程

1. 数据分发:

一个大的 mini-batch 被分成 ‘data_parallel_size’(例如 4) 份, 我们称之为 ‘D1, D2, D3, D4’。

- **第一条流水线 (节点 0)** 获得数据 ‘D1’。
- **第二条流水线 (节点 1)** 获得数据 ‘D2’。
- 以此类推。

2. 前向传播 & 后向传播:

所有流水线并行地、独立地开始工作。

- 在第一条流水线内部, 数据 ‘D1’ 被切分成微批次, 按照 PP 流水线并行的方式, 流过 ‘GPU0 -> GPU1 -> GPU2 -> GPU3’, 并完成前向和后向传播。
- 同时, 在第二条流水线内部, 数据 ‘D2’ 也以同样的方式流过它的四个 GPU。
- 这个阶段, 不同流水线之间**没有任何通信**。

3. 梯度同步 (关键步骤):

- 当一条流水线完成了一个完整 mini-batch(例如 ‘D1’) 的后向传播后, 其上的每个 GPU 都计算出了对应模型分片的**本地梯度**。
- 此时,**数据并行**的同步机制启动。
- **DP Group 0**(包含节点 0 的 GPU0, 节点 1 的 GPU0,) 中的所有 GPU 执行一次 ‘All-Reduce’ 操作, 将它们各自计算出的关于模型第一部分的梯度进行平均。
- **DP Group 1**(包含所有 GPU1) 也执行 ‘All-Reduce’, 同步模型第二部分的梯度。

- 以此类推。

这个 ‘All-Reduce’ 操作是在不同节点之间进行的, 通常通过高速网络 (如 InfiniBand) 完成。

4. 参数更新:

- 梯度同步完成后, 每个 GPU 上都有了**全局平均梯度**。
- 每个 GPU 使用这个梯度来更新它本地存储的那部分模型参数。

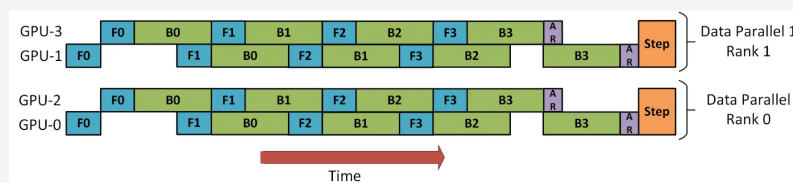


图 10.21: DP+PP 示意图

10.5.2. 3D 并行 (DP+PP+TP)

为什么需要 3D 并行？2D 并行的瓶颈

在我们讨论 DP+PP 时, 我们已经解决了一个核心问题: 如何训练一个**大到单个节点都装不下的模型**, 并将其扩展到多个节点。但这还不够, 因为 DP+PP 组合本身也存在瓶颈:

1. **流水线气泡依然存在**: 即使结合了 DP, 每条流水线 (PP) 内部的 “气泡” 开销依然是其固有效率损失。为了减少气泡, 我们需要增加微批次数量, 但这会影响模型的收敛性 (小批量噪声问题) 和硬件效率 (小计算量的 GEMM 操作效率低)。
2. **单层计算/内存瓶颈**: 想象一个流水线阶段 (Stage)。虽然整个模型被切分了, 但这个阶段本身可能仍然非常巨大。特别是对于某些 Transformer 层, **单个自注意力 (Self-Attention) 或前馈网络 (FFN) 层的权重矩阵和激活值**, 可能就大到无法放入单个 GPU。PP 是按层 (或层块) 切分, 它无法解决单层内部的瓶颈。

这就是**张量并行 (TP)** 发挥关键作用的地方。TP 可以在一个操作 (如矩阵乘法) 的内部进行并行计算, 从而解决单层过大的问题。

3D 并行的架构: 一个三维计算立方体

3D 并行架构可以用 “立方体” 结构来直观想象: 每个 GPU 都有一个 ‘(dp_rank, pp_rank, tp_rank)’ 的坐标。

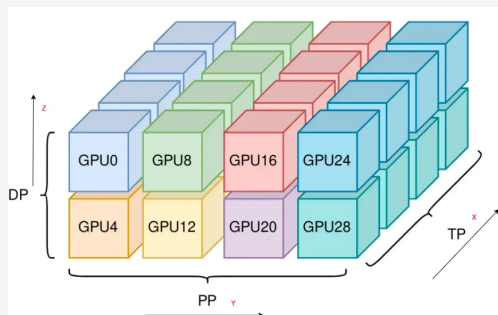


图 10.22: 3D 并行示意图

1. 维度 1: 流水线并行 (PP) - 模型的“深度”

- 它将模型的层 (layers) 沿着这个维度进行切分。
- GPU ‘(x, 0, z)’ 执行模型的第 1 个阶段, ‘(x, 1, z)’ 执行第 2 个阶段, 以此类推。
- 通信发生在 **PP 组** 内, 即 ‘pp_rank’ 不同的 GPU 之间 (例如, 从 ‘(x, 0, z)’ 到 ‘(x, 1, z)’)。

2. 维度 2: 张量并行 (TP) - 模型的“宽度”

- 它将模型中**每一层的计算** (特别是巨大的权重矩阵) 沿着这个维度进行切分。
- GPU ‘(0, y, z)’ 和 ‘(1, y, z)’ 协同工作, 共同完成一个层的计算。
- 通信发生在 **TP 组** 内, 即 ‘tp_rank’ 不同的 GPU 之间。这种通信非常频繁, **必须使用最高速的互联技术**, 因此 TP 通常被限制在单个节点内部。

3. 维度 3: 数据并行 (DP) - 模型的“数量”

- 它将整个 “PP+TP” 组合出的模型复制多份, 每份处理不同的数据。
- GPU ‘(x, y, 0)’ 和 ‘(x, y, 1)’ 持有完全相同的模型分片, 但处理不同的数据批次。
- 通信发生在 **DP 组** 内, 即 ‘dp_rank’ 不同的 GPU 之间。这种通信是梯度同步, 通常跨节点。

总 GPU 数量 = ‘data_parallel_size’ * ‘pipeline_parallel_size’ * ‘tensor_parallel_size’

一个具体的例子: 用 64 个 GPU 进行 3D 并行

假设我们有 8 个节点, 每个节点 4 个 GPU, 总共 32 个 GPU。我们可以这样配置:

- ‘tensor_parallel_size = 4’: 使用每个节点内的 4 个 GPU 进行张量并行。
- ‘pipeline_parallel_size = 4’: 将模型切分为 4 个流水线阶段。
- ‘data_parallel_size = 2’: 复制出 2 个数据并行的副本。

一个 GPU 的角色：让我们看看 ‘GPU9’ (全局排名第 9 的 GPU) 在做什么。

- 它位于**节点 1**上 (‘GPU8’到 ‘GPU11’在节点 1)。
- 它的**TP 组**是节点 1 上的所有 4 个 GPU (‘GPU8’到 ‘GPU11’)。它们共同负责计算模型中的每一层。
- 它的**PP 阶段**是什么？假设我们将 4 个 PP 阶段分配给 4 组节点，那么节点 0 和 1 可能构成一个数据并行副本，节点 2 和 3 构成另一个。那么 ‘GPU9’位于 PP 阶段 2。
- 它的**DP 组**是哪个？它会和另一个数据副本中处于相同 TP 和 PP 位置的 GPU 构成 DP 组。

工作流程一览：

1. **输入数据 ‘D’**被分成两部分 ‘D1’ 和 ‘D2’，分别送往两个数据并行副本。
2. **在数据副本 1 中：**
 - ‘D1’被切成微批次。
 - **阶段 0 (例如，由节点 0 的 4 个 GPU 负责)：**这 4 个 GPU 通过**张量并行**协同计算模型的前 $\frac{1}{4}$ 的层。计算完成后，将激活值发送给下一阶段。
 - **阶段 1 (例如，由节点 3 的 4 个 GPU 负责)：**接收来自阶段 0 的激活值，通过**张量并行**计算模型的第二个 $\frac{1}{4}$ 的层。
 - 以此类推，完成整个流水线的前向和后向传播。
3. 在**数据副本 2**中，完全相同的过程用数据 ‘D2’并行地发生。
4. **梯度同步：**
 - 在每个副本完成后，处于相同 ‘pp_rank’和 ‘tp_rank’的 GPU 之间进行 ‘All-Reduce’。
 - 例如，节点 0 上的 ‘GPU0’与节点 1 上的 ‘GPU4’进行梯度同步。

10.5.3. 4D 并行 (DP+PP+TP+ZeRO)

为什么需要这个组合？

我们已经知道 3D 并行 (DP+PP+TP) 非常强大，但它仍然有一个根本性的“浪费”：在数据并行的维度上，不同的副本 (Data Parallel Replicas) 之间存在着**完全相同的模型状态冗余**。

例如，在一个 ‘dp_size=8, pp_size=4, tp_size=8’ 的配置中：

- 由 PP 和 TP 组合后，每个 GPU 只负责整个模型 ‘ $1/(4*8) = 1/32$ ’ 的计算和参数。
- 但是，这个 ‘ $1/32$ ’ 的模型分片，在 8 个数据并行副本中是**一模一样的**。这意味着，我们依然为参数、梯度和优化器状态多占用了 7 倍的内存。

对于一个万亿参数的模型，即使是 ‘ $1/32$ ’ 的分片也可能非常巨大。消除这最后的冗余，就是 ZeRO 在这个组合中要完成的使命。

架构和 workflow: ZeRO 如何融入 3D 并行

ZeRO 的操作完全发生在数据并行组 (DP Group) 内部。因此一样用立方体看待。

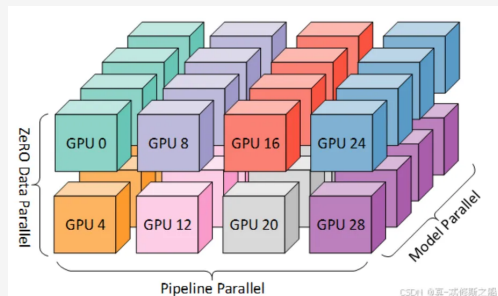


图 10.23: 4D 并行示意图

状态分区

考虑一个特定的模型块, 它已经被 PP 和 TP 切分好了, 位于坐标 $(pp_rank=p, tp_rank=t)$ 的位置上。这个模型块的状态 (参数、梯度、优化器状态) 需要被存储。在传统的 3D 并行中, 所有 dp_rank 从 0 到 dp_size-1 的 GPU, 在 (p, t) 这个位置上, 都存储着一份完整的、相同的模型块状态。

引入 ZeRO 后: 这个模型块的状态被进一步切分成 dp_size 份, 分散存储在多个 DP 组中。

- GPU $(dp=0, pp=p, tp=t)$ 只存储这份状态的第 1 片。
- GPU $(dp=1, pp=p, tp=t)$ 只存储这份状态的第 2 片。
- 以此类推。

训练步骤的变化

这个改变深刻地影响了训练流程, 尤其是在 DP 组内的通信模式:

● 前向传播 (ZeRO-Stage 3):

1. 在一个 GPU (d, p, t) 开始计算它的模型块之前, 它只拥有该模型块 $1/dp_size$ 的参数。
2. 它必须先在它的 DP 组内发起一次 'All-gather' 操作。
3. 通过这次通信, 它从 DP 组的其他伙伴那里收集齐了完整的模型块参数。
4. 然后, 它和它的 TP 组内的伙伴们一起, 协同完成这一层的前向计算。
5. 计算完成后, 为了节省内存, 它可以立即丢弃刚刚收集来的那部分不属于自己的参数。

● 后向传播:

1. 与前向传播类似, 在计算梯度前, 需要通过 'All-gather' 确保拥有完整的参数。
2. 计算完成后, 每个 GPU 都得到了它负责的模型块的部分梯度 (因为 TP)。

- TP 组内部会进行一次 ‘All-Reduce’, 使得 TP 组内每个 GPU 都拥有了关于这个模型块的**完整梯度**。

● **梯度同步与分区 (取代传统 DP 的 All-Reduce):**

- 现在, DP 组内的每个 GPU 都有一份完整的梯度。
- 它们不再执行简单的 ‘All-Reduce’ 来求平均。
- 取而代之, 它们执行一次 ‘Reduce-Scatter’。这个操作会一边将所有 DP 副本的梯度相加求平均, 一边将结果**直接分区**, 并发送给对应的 GPU。
- 操作结束后, GPU ‘(d, p, t)’ 只会收到它所负责的、平均后的**梯度的一小片**。

● **参数更新:**

- 每个 GPU 使用它本地存储的一小片优化器状态和刚刚收到的一小片梯度, 来更新它本地拥有的一小片模型参数。

10.5.4. 专家并行/MOE 并行

和前面的并行几乎完全不同, 专家并行只针对推理阶段, 它的思想是只利用和输入 token 最相关的几个专家模型 (top-k) 来进行推理最后加权输出。

但模型本身会有很多个专家来“顾问”, 每次只用几个, 来这样实现所谓”并行”。

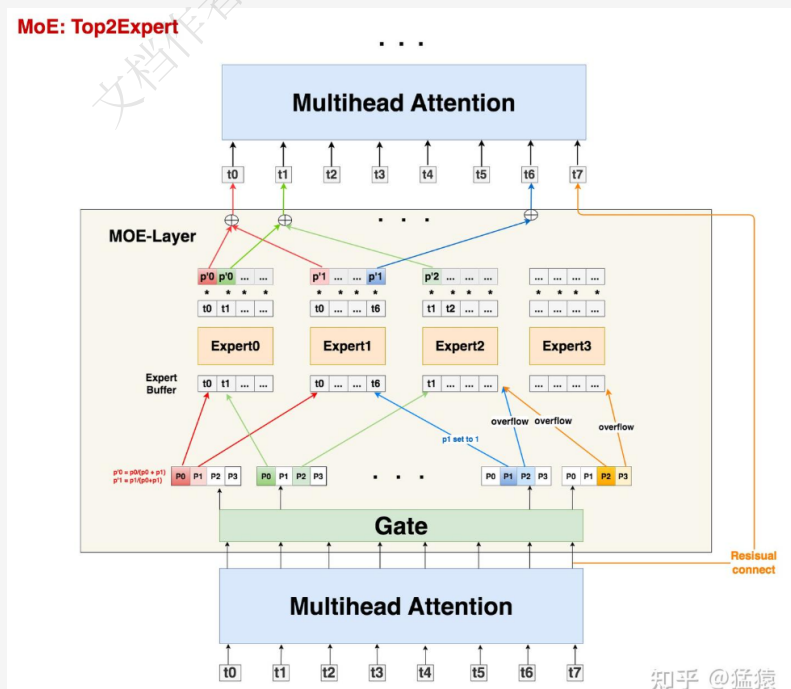


图 10.24: 专家并行示意图

10.6. CUDA 通信机制知识补充

核心原则: 异构计算

Keynote 10.3

异构计算是指结合**不同指令值和体系结构的设备 (计算单元)**之间的联合计算, 比如 *CPU*、*GPU*、*FPGA*。在 *CUDA* 编程中主要就是指 *CPU* 及内存与 *GPU* 和显存。这两者在**物理上独立, 通过 *PCIe* 总线连接通信**。

Keynote 10.4

PCIe 通信特点

- **连接方式**: *PCIe* 采用**点对点的串行连接**方式, 每个设备都有自己的**专用连接**, 可以独享带宽, 无需像 *PCI* 这种旧的并行总线那样共享同一个物理带宽。
- **拓扑结构**: *PCIe* 系统是一个**树状的层次结构**, 通常由代表 *CPU* 接口的根复合体、用于扩展端口的交换器 (*Switch*) 以及终端设备 (*Endpoint*, 如显卡、网卡、固态硬盘等) 组成。
- **分层架构**: *PCIe* 协议采用**分层架构**, 分为事务层、数据链路层和物理层。
 - **事务层**: 负责生成和解析用于数据请求和响应的事务层数据包 (*TLP*)。
 - **数据链路层**: 确保数据传输的完整性, 负责错误检测和纠正。
 - **物理层**: 负责数据的串行化传输和链路管理。

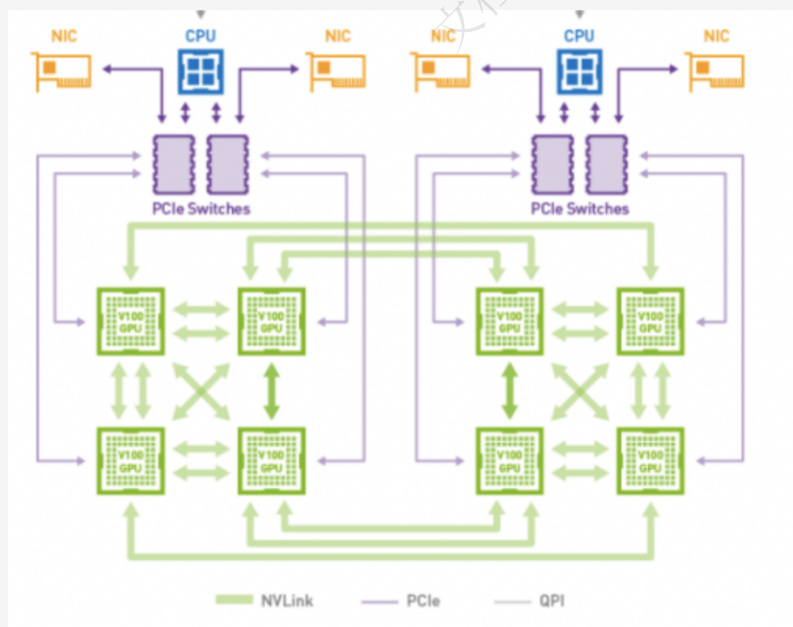


图 10.25: CUDA 通信机制示意图

CUDA 里由于 CPU 与 GPU 物理上的分离使得很有必要研究一下两者之间的通信。这也带来了 GPU 主要的性能瓶颈: 通信与计算之间的不平衡, 即**数据传输的**

速度远低于 GPU 内部的计算速度

CUDA 中的通信可以分为两大类：

1. **主机与设备 (Host-Device) 之间的通信**：宏观层面的数据传输。
2. **设备内部 (Intra-Device) 的通信**：微观层面，线程之间的数据交换。

10.6.1. 主机与设备 (Host-Device) 通信

这是最基础的数据交互。典型的流程是：

1. CPU 将数据从主存复制到显存。
2. GPU 执行计算 (Kernel Launch)。
3. CPU 将结果从显存复制回主存。

同步 (Synchronous) 内存拷贝

特性：

- **阻塞式 (Blocking)**：当 CPU 线程调用拷贝函数时，它会**暂停执行 (阻塞)**，直到数据拷贝**完全完成**，然后才会继续执行后续的 CPU 代码。
- **简单易懂**：逻辑清晰，不易出错。
- **性能瓶颈**：CPU 在等待期间完全闲置，无法与 GPU 的数据传输并行工作，浪费了宝贵的计算资源。

异步 (Asynchronous) 内存拷贝与流 (Streams)

为了克服同步拷贝的性能瓶颈，CUDA 引入了**异步拷贝**和**流 (Stream)**的概念。

流 (Stream)：可以理解为 GPU 上的一个**任务队列**。你向一个流中添加一系列操作 (如内存拷贝、核函数启动)，GPU 会按照你添加的顺序来执行这些操作。

异步拷贝：

- **非阻塞式 (Non-Blocking)**：当 CPU 线程调用异步拷贝函数时，它只是将这个“拷贝任务”**提交到指定的流**中，然后**立即返回**，继续执行后续的 CPU 代码。CPU 不会等待拷贝完成。
- **并行潜力**：这使得 CPU 可以继续准备下一个数据块、启动其他任务，或者让数据传输与 GPU 计算**重叠 (Overlap)**执行，从而极大地提升整体效率。

Keynote 10.5

关键点:

- **异步操作必须指定一个流。**
- **要使异步内存拷贝真正实现非阻塞, 主机内存必须是页锁定 (Page-Locked) 或固定 (Pinned) 内存。**因为操作系统可能会移动常规的可分页内存, 而 GPU 的 DMA 引擎需要一个固定的物理地址。使用 `'cudaMallocHost()'` 或 `'cudaHostAlloc()'` 分配页锁定内存。
- **需要显式同步。**既然 CPU 不等待, 那你如何知道 GPU 任务何时完成? 你需要使用同步函数, 如 `'cudaStreamSynchronize()'` (等待指定流完成) 或 `'cudaDeviceSynchronize()'` (等待设备上所有任务完成)。

10.6.2. 设备内部 (Intra-Device) 通信

当数据已经在 GPU 上后, 成千上万的线程需要相互协作来完成计算。它们的通信方式和范围是严格分级的。

线程块 (Block) 内部通信

这是最高效、最常用的设备内部通信方式。同一线程块内的线程可以**共享数据和进行同步**。

共享内存 (Shared Memory):

- **物理位置:** 位于 GPU 芯片上 (On-chip), 与 L1 缓存共享物理资源。
- **速度:** 访问速度**极快**, 远高于全局内存 (Global Memory), 几乎与寄存器 (Register) 访问速度相当。
- **生命周期:** 与线程块相同。当线程块开始执行时分配, 执行完毕后释放。
- **作用域:** 仅对同一线程块内的所有线程可见。块 A 的线程无法访问块 B 的共享内存。
- **声明:** 使用 `'__shared__'` 限定符在核函数内部声明。

同步原语 (`'__syncthreads__()'`):

- **功能:** 这是一个**栅栏 (Barrier)**。块内的所有线程执行到 `'__syncthreads__()'` 时必须停下来等待, 直到**块内所有线程**都到达这个点, 然后才能继续执行后面的指令。
- **必要性:** 确保数据依赖性。例如, 在线程 A 从共享内存读取数据之前, 必须确保所有向该共享内存写入数据的线程 (如线程 B、C、D) 都已经完成了写入操作。 `'__syncthreads__()'` 就是用来保证这一点的。

这是一个更复杂的问题。CUDA 的编程模型**没有提供直接、高效的块间同步机制**。不同块的执行顺序和并发性由硬件调度器决定, 程序员无法假设块 A 一定在块 B 之前完成。

因此, 块间通信通常通过**全局内存 (Global Memory)**间接实现:

1. 分步执行 (Separate Kernels):

- 这是最常用、最可靠的方法。
- 流程:
 - a. 启动一个核函数 (Kernel 1), 每个块完成其部分计算, 并将中间结果写入全局内存。
 - b. 核函数结束后, 有一个隐式的全局同步。
 - c. 启动第二个核函数 (Kernel 2), 读取 Kernel 1 产生的中间结果, 进行下一阶段的计算。
- 缺点: 多次核函数启动会带来额外的开销。

2. 原子操作 (Atomic Operations):

- 当多个块中的线程需要更新**同一个全局内存地址**时 (例如, 一个全局计数器), 为了避免数据竞争, 必须使用原子操作。
- 函数: 'atomicAdd()', 'atomicExch()', 'atomicCAS()' 等。
- 特性: 保证操作的原子性 (不会被中途打断), 但可能会因为争用而导致线程串行化, 影响性能。

3. 持久化线程 (Persistent Threads) 与网格级同步 (Grid-Level Sync) (高级)

- 对于一些复杂的应用 (如图计算、某些稀疏计算), 可以启动一个覆盖整个设备的持久化网格 (Grid), 线程在循环中不断从全局队列中获取任务。
- 较新的 CUDA 版本 (计算能力 7.x+) 引入了 **Cooperative Groups**, 提供了在整个 Grid 范围内同步的机制 ('grid.sync()')。这允许在**单个核函数内**实现全局同步, 避免了多次启动的开销。但这需要谨慎设计, 且对硬件有要求。

11.1. 拓展链接

- 泛函分析概论：
(<https://mp.weixin.qq.com/s/ebhG6fFNry0kqKulTzFzeg>)
-

11.1 拓展链接 153

11.2 经典反例 154

11.3 其他 155

文档作者：喂喂薇

11.2. 经典反例

文档作者：喂喂薇

11.3. 其他

文档作者：喂喂薇